

SepLLM：通过将一段内容压缩为一个分隔符来加速大语言模型

陈国轩^{1,2} 施涵¹ 李家伟¹ 高一航² 任晓哲¹ 陈一萌³ 蒋鑫¹
李振国¹ 刘伟阳⁴ 黄超²

项目主页: sepllm.github.io

大语言模型 (LLMs) 在一系列自然语言处理任务中展现出了卓越的性能。然而, 由于其二次复杂度, 其庞大的规模带来了巨大挑战, 特别是在计算需求和推理速度方面。在这项工作中, 我们发现了一个关键模式: 与具有语义意义的词元相比, 某些看似无意义的分隔符词元 (即标点符号) 对注意力分数的贡献不成比例。这一观察表明, 这些分隔符之间的片段信息可以有效地压缩到分隔符词元本身, 而不会造成显著的信息损失。基于这一洞见, 我们提出了 SepLLM, 一个即插即用的框架, 通过压缩这些片段并消除冗余词元来加速推理。此外, 我们还实现了用于加速训练的高效内核。在免训练、从头训练和后训练设置下的实验结果证明了 SepLLM 的有效性。值得注意的是, 使用 Llama-3-8B 骨干网络, SepLLM 在 GSM8K-CoT 基准测试上实现了超过 50% 的 KV 缓存减少, 同时保持了可比的性能。此外, 在流式处理设置中, SepLLM 能够有效处理长达 400 万词元或更长的序列, 同时保持稳定的语言建模能力。

1. Introduction

«<LEX_CMD₂>>>

基于 Transformer 的模型 (Vaswani et al., 2017) 在广泛的任务中展现出卓越的性能, 包括自然语言处

¹ 华为诺亚方舟实验室 ² 香港大学 ³ 阿卜杜拉国王科技大学生成式人工智能卓越中心 ⁴ 马克斯·普朗克智能系统研究所, 蒂宾根. Correspondence to: 施涵 <shi.han@huawei.com>.

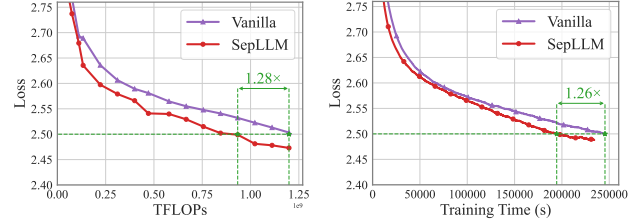


Figure 1. 原始 Transformer 与所提出的 SepLLM 之间的损失对比。在不同计算成本和不同训练时间下, SepLLM 始终能获得更低的损失。

理 (Zhang et al., 2020; Raffel et al., 2020)、计算机视觉 (Dosovitskiy 等人, 2020) 以及科学机器学习 (Geneva & Zabarar, 2022)。然而, 依赖下一词预测的原始 Transformer 面临着显著的计算挑战, 尤其是在扩展到更大模型和更长上下文时。这些计算低效率严重影响了推理速度和训练时间。

这些效率问题的核心挑战在于自注意力模块, 其计算复杂度相对于输入词元数量呈二次方增长。关于大语言模型中高效 Transformer 的研究主要遵循两个主要方向。第一种方法侧重于线性注意力 (Katharopoulos 等人, 2020; Schlag et al., 2021), 用具有线性复杂度的替代方案替换原始的自注意力模块。然而, 这些修改使得架构与传统自注意力有显著不同, 阻碍了直接利用强大的预训练 Transformer 模型。第二种方法强调 KV 缓存优化 (Xiao et al., 2024a; Zhu et al., 2024; Xiao et al., 2024b; Li 等人, 2024), 旨在消除冗余的 KV 缓存以适应更长的输入上下文。例如, Xiao et al. (2024a) 引入了一种自适应机制, 根据累积注意力分数选择性地保留必要的词元及其 KV。类似地, Zhu et al. (2024) 提出了一种具有可控稀疏性的词元选择策略, 实现了近乎无损的加速。尽管前景广阔, 但这些免训练方法难以适应训练阶段, 导致训练与推理性能之间存在差异。

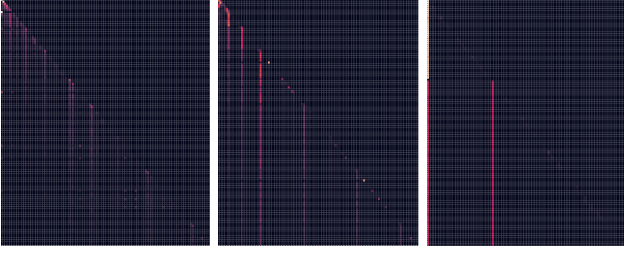


Figure 2. 给定输入 “Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. ...” 时，不同层的注意力分数可视化。请注意，分隔符词元如 “,” 和 “.” 贡献了大量的注意力。

StreamingLLM (Xiao et al., 2024b) 代表了一项值得注意的尝试，它通过保留注意力汇聚点和局部词元来减少计算和内存开销，以解决这些限制。然而，它省略了许多中间词元，导致与标准 Transformer 相比性能有所下降。

为了更好地理解大语言模型的内在机制，我们分析了不同样本的注意力模式。图 2 展示了当 Llama-3-8B-instruct (Dubey 等人, 2024) 处理一个数学问题时的注意力分布（完整结果见附录 A）。令人惊讶的是，大语言模型倾向于将注意力优先分配给看似“无意义”的分隔符词元（如 “.” 或 “\n”），而不是关注具有语义意义的词元（如名词和动词）以进行信息检索。这一观察表明，片段信息被压缩并嵌入到这些分隔符词元中，从而无需直接从内容词元中提取即可实现高效的信息检索。受此观察启发，我们提出了 SepLLM，这是一种新的语言建模视角，也是一种高效的 Transformer 架构，其采用了一种数据依赖的稀疏注意力机制，该机制选择性地仅保留初始、相邻和分隔符标记，同时丢弃其他标记。无需训练的 SepLLM 性能与原始 Transformer 相当，这验证了我们的假设：片段信息被有效地压缩到了分隔符标记中。

更重要的是，我们将 SepLLM 集成到训练阶段（包括从头训练或微调），并基于 FlexAttention (PyTorch, 2024) 实现了一个硬件高效的内核。这种集成减少了先前方法中存在的训练与推理之间的差异。

如图 1 所示，在相同的计算成本或训练时间下，SepLLM 始终比原始 Transformer 获得更低的损失。

此外，在达到相同训练损失的情况下，SepLLM 将计算成本降低了 28%，训练时间减少了 26%。我们的贡献总结如下：

- 我们通过可视化标记级注意力分数来分析注意力模式，发现初始、相邻和分隔符标记始终获得较高的注意力权重。这些实证发现促使我们提出 SepLLM、a new language modeling perspective 以及一个简单而有效的框架来加速推理。
- 通过对训练良好的大语言模型进行有针对性的掩码实验，我们证明分隔符标记包含关键信息，对模型性能至关重要。这一发现表明，序列最初由分隔符分割，片段信息被压缩到这些频繁被关注的分隔符标记中，而冗余的具体标记可以被丢弃。
- 我们进行了全面的实验，在各种任务、数据集和骨干模型上验证 SepLLM 的有效性，考察了其在免训练、从头训练和后训练设置下的性能。
- 我们已将实现公开在 sepllm.github.io。我们的代码库支持高效的多节点分布式训练，带有加速的注意力模块 Sep-Attention，并且还支持众多现有的融合算子来加速训练过程，例如 fused rope (Su et al., 2024)、fused layer norm 等。

2. Related Work

KV Cache Compression. 近期研究集中于克服大型语言模型在处理长上下文输入时的局限性。FastGen (Ge 等人, 2024) 提出了一种自适应的键值缓存管理方法，通过为不同注意力头定制保留策略来优化内存使用。SnapKV (Li 等人, 2024) 通过键值缓存压缩提升效率，利用注意力分数选择并聚类重要位置。H₂O (Zhang et al., 2023) 实现了一种动态令牌保留策略，平衡近期与历史重要信息以优化内存使用。StreamingLLM (Xiao et al., 2024b) 扩展了大型语言模型处理无限长序列的能力而无需微调，其方法是通过保留注意力汇聚点与局部令牌。QuickLLaMA (Li 等人, 2025) 提出驱逐查询感知的键值缓存以加速推理。PyramidInfer (Yang et al., 2024) 与 PyramidKV (Zhang et al., 2024) 在不同层间调整键值缓存容量，优先在较低层分配较大容量，同

时减少较高层的分配。然而，此类工作大多无法应用于训练阶段。

Sparse Attention. 稀疏注意力涉及通过将注意力限制在预定义模式（如局部窗口或固定步长的块模式）来创建稀疏注意力矩阵。Beltagy et al. (2020) 将扩张的局部窗口注意力与任务特定的全局注意力相结合。BigBird (Zaheer et al., 2020) 提出了一种线性复杂度的注意力替代方案，使用全局令牌、局部滑动窗口注意力和随机注意力。相比之下，SparseBERT (Shi et al., 2021) 提出了一种可微分的注意力掩码算法，以端到端的方式学习注意力掩码。需注意，大多数关于稀疏注意力的工作使用固定掩码并基于 BERT (Devlin 等人, 2019) 系列模型构建。相比之下，我们提出的 SepLLM 主要基于 GPT (Brown 等人, 2020) 系列构建，且其注意力掩码是数据依赖的。

3. Method

3.1. Fundamental Design

从图 2 中我们可以观察到，在给定的输入上下文中，看似“无意义”的分隔符相较于具有实际语义的标记获得了更高的注意力分数。因此，我们提出了一种新颖的 Transformer 架构，其中对于 Transformer 的某一层（即自注意力层），输入中的每个标记只能看到前一个 Transformer 层输出的、位于当前标记之前的标记的部分（而非全部）隐藏状态。这个标记子集包括一定数量的起始词（例如，注意力汇聚点 (Xiao et al., 2024b)）、当前标记之前的所有分隔符标记，以及最接近当前标记的 n 个标记。具体细节如下。

Initial Tokens. 当使用滑动窗口机制 (Beltagy et al., 2020) 进行生成时，从键值 (KV) 缓存中移除与起始标记对应的键值会导致生成标记的困惑度显著增加，这一现象在 Xiao et al. (2024b) 中有所提及。起始的几个标记也被称为注意力汇聚点。我们保留了这一设置，并在后续实验中进一步验证了起始标记的作用。通常，会保留 a 个起始标记。

Separator Tokens. 从图 2 中我们可以观察到，在

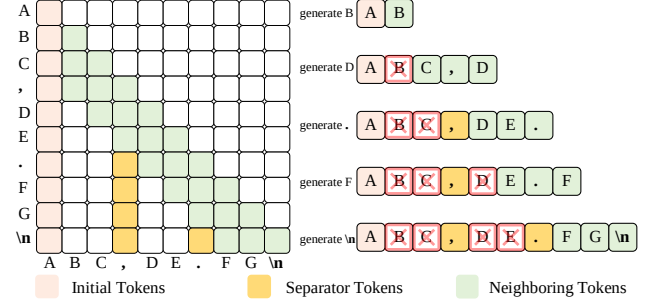


Figure 3. SepLLM 的整体范式。左侧展示了在给定输入“ABC,DE.FG\n”时，训练或预填充阶段的注意力掩码。右侧说明了生成阶段的 KV 缓存管理。

给定的输入上下文中，用于分割序列的看似“无意义”的分隔符（如逗号、句号、感叹号、分号等）相较于具有语义意义的标记（如名词或动词）获得了更高的注意力分数。因此，我们假设这些分隔符可能压缩了由它们自然分割的文本片段的信息，使得 Transformer 在生成新标记时，只需参考这些分隔符所包含的信息即可提取出与那些文本片段相关的信息。因此，在无需训练的场景下，我们采用了这一策略，并在许多任务上取得了与基于完全注意力的原始模型相似的结果。此外，为了强化使用分隔符压缩其各自片段内信息的效果，我们采用了从头训练和训练后调整的方法，迫使模型在训练过程中限制当前标记访问来自远处前文的所有信息，即，在每个片段中，只有代表该片段的分隔符对当前标记可见（其他标记被掩码，见图 3）。经过这种方式训练后，片段内的信息被迫压缩到分隔符中，使得 Transformer 预测下一个词的概率分布与具有完全注意力的原始 Transformer 非常接近。更多细节参见附录 H 和 I。

Neighboring Tokens. 语言任务通常表现出强烈的局部依赖性和交互性，因为相邻的标记常常形成连贯的短语或具有需要捕捉的依赖关系。邻近的标记通常有助于形成局部平滑且连贯的上下文，使模型能够生成在即时上下文中合理的句子。邻近标记，也称为局部注意力或滑动窗口注意力，在各种高效 Transformer 中均有考虑 (Xiao et al., 2024b; Zhang et al., 2023)，我们也采用了这种方法，将最接近当前标记的前序标记数量记为“ n ”。

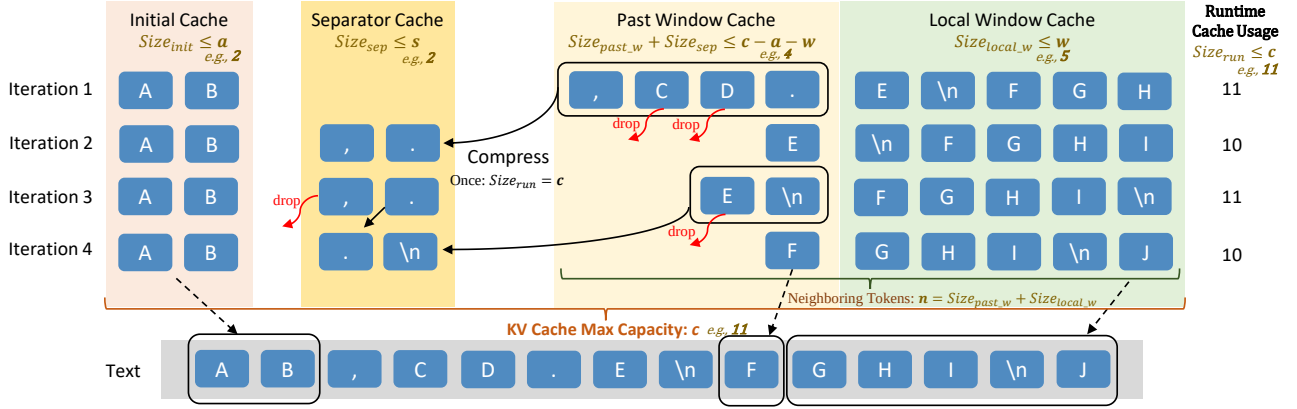


Figure 4. 为流式应用定制的所提 SepLLM 的整体框架。KV 对被存储在四个缓存块中（显示为四列），并在每次迭代中更新（以单行显示）。一旦运行时使用量 $Size_{run}$ 达到最大容量 c ，SepLLM 会将过去窗口缓存中分隔符令牌牌的 KV 缓存移至分隔符缓存，并丢弃其他 KV 缓存。

3.2. Overall Pipeline

我们将所提出的 SepLLM 的整体流程划分为训练/预填充阶段和生成阶段。我们还在附录 J 和 K 中提供了关于 SepLLM 的 Universal Approximation 的理论分析。

Training/Pre-filling. 在 SepLLM 架构的训练/预填充阶段，我们不需要将输入上下文中所有标记对应的查询向量与所有键向量相乘。仅需计算掩码矩阵（如图 3 所示）中高亮元素对应的查询-键对向量之间的乘法即可。其公式化表示如下。

$$\begin{aligned} A &= \text{Softmax}(\Lambda), \Lambda = \frac{\text{Mul}(Q, K^T | M)}{\sqrt{d_k}} \\ O &= A \cdot V \end{aligned} \quad (1)$$

其中， $Q \in \mathbb{R}^{m \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$ 是单个注意力层的查询矩阵和键矩阵，其中每个行向量 Q_i, K_j 对应于序列长度为 m 的输入上下文中第 i 个标记的查询和第 j 个标记的键。 d_k 表示键向量和查询向量的维度。 $\Lambda, A \in \mathbb{R}^{m \times m}$ 分别是原始注意力图和最终注意力图。 $V \in \mathbb{R}^{m \times d_v}$ 是维度为 d_v 的值矩阵， $O \in \mathbb{R}^{m \times d_v}$ 表示当前注意力层的输出。 $\text{Mul}(\cdot)$ 表示一个稀疏矩阵乘法函数，可通过诸如 Zhu et al. (2024) 等方法进行优化，并且我们也实现了名为 Sep-Attention 的自定义模块来加速此过程。 $M \in \mathbb{B}^{m \times m}$ 是一个二元

掩码矩阵¹，用作 $\text{Mul}(\cdot)$ 的参数：

$$\Lambda_{i,j} = \begin{cases} Q_i^T K_j / \sqrt{d_k}, & \text{if } M_{i,j} = 1 \\ -\infty, & \text{if } M_{i,j} = 0 \end{cases} \quad (2)$$

其中， $\Lambda_{i,j}, A_{i,j}, M_{i,j}$ 分别是矩阵 Λ, A, M 中第 i 行、第 j 列的元素。由于当 $\Lambda_{i,j} = -\infty$ 时 $A_{i,j} = 0$ ，那些不是 Initial、Separator 和 Neighboring 标记的标记将在公式 1 中被 $A \cdot V$ 掩蔽。此策略（公式 1）适用于多头注意力 (Vaswani et al., 2017) 的所有头。

Generation. 在此 Fundamental Design（第 3.1 节）的生成阶段，KV 缓存的管理也非常直观。如图 3 右侧所示，在生成新标记时，我们仅保留 Initial、Separator 和 Neighboring 标记的 KV 缓存。因此，所提出的 SepLLM 中的 KV 缓存要小得多，所需内存也更少。理想情况下，基于 SepLLM，生成下一个词的困惑度与使用完全注意力的原始 Transformer 相当。

3.3. Tailored Streaming Design

在现实场景中，存在许多流式应用，例如多轮对话，其中预期会有长交互 (Xiao et al., 2024b)。因此，我们期望 SepLLM 能够处理无限长的输入，而不会显著牺牲效率和性能，尤其是在流式应用中。如 Fundamental design（第 3.1 节）所述，SepLLM 通过仅保留分隔符、相邻及起始标记的 KV 缓存，可

¹ $\mathbb{B} := \{0, 1\}$ ，这是一个二元集合。

以节省大量 KV 缓存。然而，随着输入标记数量的增加，KV 缓存中的分隔符数量也将无限累积，这对于流式设置是不可行的。因此，我们针对流式场景提出了 Tailored Streaming Design。

Framework. 图 4 展示了 SepLLM 在流式应用中的处理架构。该图描绘了多次迭代，每一行代表一个独立的处理步骤。系统同时维护四个专用的缓存块：初始缓存、分隔符缓存、过往窗口缓存和局部窗口缓存。具体而言，初始缓存捕获了 Xiao et al. (2024b) 提出的注意力沉没点。局部窗口和过往窗口缓存存储连续标记的 KV，其中过往窗口缓存充当局部窗口缓存的溢出缓冲区。分隔符缓存保留分隔符的 KV，这些分隔符包含了压缩后的片段信息。为描述缓存管理策略，我们将四个缓存的运行时使用量分别记为 $Size_{init}$ 、 $Size_{sep}$ 、 $Size_{past_w}$ 和 $Size_{local_w}$ 。所有 KV 缓存的运行时使用量定义为 $Size_{run} := Size_{init} + Size_{sep} + Size_{past_w} + Size_{local_w}$ ，其满足 $Size_{run} \leq c$ 。连续相邻标记的数量定义为 $n := Size_{past_w} + Size_{local_w}$ 。值得注意的是， n 是输入序列长度 m （以及特定输入数据集 $\mathcal{D}$ ）的函数，而非流式设置中的一个固定超参数。为清晰起见，我们将此缓存系统的预设超参数详述如下（Note: $a + s + w < c$ ）。

- c : 整个 KV 缓存的最大容量。
- a : 初始缓存的最大容量。
- s : 分隔符缓存的最大容量。
- w : 局部窗口缓存的最大容量。值得注意的是， w 也是运行时 KV 缓存使用量 $Size_{run}$ 首次达到 c 后， n 的最小值。

在流式序列生成过程中，SepLLM 将首先填充初始缓存和局部窗口缓存。当 $Size_{local_w}$ 达到 w 后，后续的令牌将被导向过去窗口缓存。当 $Size_{run}$ 达到 c 时（图 4 中的迭代 1），将触发压缩，此时过去窗口缓存中的分隔符令牌被移至分隔符缓存，其他令牌则被丢弃。

当分隔符缓存在某个总输入长度 m_0 处达到其容量 s 时， n 进入周期性模式。具体而言，对于 $m > m_0$ ， n 遵循一个由 w 和 $c - a - s$ 界定的周期性线性函

数。KV 缓存的详细演变过程在附录 B 中说明。为了分析 KV 缓存的平均使用情况，我们定义 $\bar{n}_m := \frac{1}{m} \sum_{k=1}^m n(k)$ 。根据线性和周期性，我们有

$$\lim_{m \rightarrow \infty} \bar{n}_m = \frac{w + c - a - s}{2}. \quad (3)$$

对于无限长序列生成的平均运行时 KV 缓存使用量，我们有

$$\begin{aligned} \lim_{m \rightarrow \infty} \overline{Size_{run}} &= \lim_{m \rightarrow \infty} \bar{n}_m + a + s \\ &= \frac{w + c + a + s}{2} < c. \end{aligned} \quad (4)$$

Positional Encoding. 我们在流式设置中的位置编码策略与最先进的 StreamingLLM (Xiao et al., 2024b) 相同，该策略专为无限长输入设计，其中我们关注的是缓存内的位置，而非原始文本中的位置。

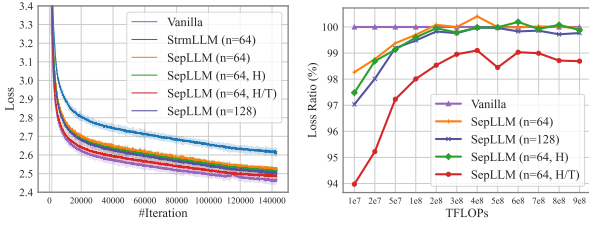
4. Experiments and Results

4.1. Experimental Settings

我们在以下任务上评估我们提出的 SepLLM，即无需训练、从头训练、后训练以及流式应用。

Model. 评估中采用了两个流行的模型系列，即 Pythia (Biderman et al., 2023) 和 Llama-3 (Dubey 等人, 2024)。具体而言，在从头训练任务中，我们使用 Pythia-160m-deduped 作为骨干模型，因为该模型的模型、数据、配置和检查点均为开源，且训练结果可复现。对于后训练设置，我们以 Pythia-1.4B-deduped 作为骨干模型。尽管 Llama-3 在各种下游任务上表现出强大的性能，但其训练实验细节并未公开。因此，我们仅将 Llama-3 用于无需训练和流式任务。

Training Datasets. 在从头训练和后训练任务中，我们使用去重后的 Pile (Gao 等人, 2020) 进行训练，该数据集包含约 2070 亿个词元。所有其他配置均与 Pythia (Biderman et al., 2023) 的相应设置保持一致。具体来说，训练轮数设置为 1.5 轮（全局批次大小为 1024，共 143,000 步），这意味着从头训练总共使用了约 3000 亿个词元，这与 Pythia (Biderman et al., 2023) 的设置相同。



(a) 损失相对于步数 (b) 损失比率相对于 FLOPs

Figure 5. 从头开始训练的损失曲线。7(b) 显示了不同方法的损失值与 Vanilla 方法损失值相对于 FLOPs 的比率。

Parameter Setting. 我们使用 Pythia-1.4B-deduped 模型的官方 93,000 步检查点来进行后训练，这对应于在去重 Pile 数据集 (Gao 等人, 2020) 上恰好完成一个训练轮次。并且，[“, “”, “?”, “!”], “;”, “:”, “ ”, “\t”, “\n”] 是所有评估中使用的分隔符词元。更具体的实验设置将在各自的实验部分介绍。

4.2. Training-Free

我们在无需训练的任务中，基于流行的 Llama-3-8B-Instruct 模型 (Dubey 等人, 2024)，对所提出的模型架构进行了评估。

Benchmarks. 我们采用了具有代表性且常用的 GSM8K-CoT (Cobbe 等人, 2021) 和 MMLU (Hendrycks 等人, 2021)。GSM8K-CoT (Cobbe 等人, 2021) 通过评估模型的推理和逐步解决问题的能力，来测试其解决数学问题的能力。CoT (Wei et al., 2022) 意味着通过将复杂问题分解为一系列逻辑步骤来模拟推理过程的能力。我们采用了默认的 8-shot 设置。MMLU (Hendrycks 等人, 2021) 评估模型在广泛学科（如历史、科学、数学等）上的通识知识和推理能力。对于 MMLU，我们使用了常用的 5-shot 设置。

Results. 无需训练的实 1 (Xiao et al., 2024b)

4.3. Training from Scratch

我们训练了原始的 Pythia-160m-deduped 模型，以及采用 SepLLM（及 StreamingLLM）架构修改后的 Pythia-160m-deduped 模型，在 Pile 数据集上进

行了 143,000 步训练，全局批次大小为 1024（训练总共涉及约 300B 个词元）。所有训练配置均与 Pythia (Biderman et al., 2023) 保持一致（参见第 4.1 节）。并且遵循 Pythia (Biderman et al., 2023)，我们在以下下游任务上进行了测试：ARC-Challenge 和 ARC-Easy (Clark 等人, 2018)、LAMBADA (Paperno 等人, 2016)（针对困惑度和准确率）、LogiQA (Liu 等人, 2021)、PIQA (Bisk et al., 2020)、SciQA (Welbl et al., 2017)。根据图 7 中描绘的损失曲线以及表 2 中的下游性能，我们得出以下分析。

Neighboring Token Benefits. 基于设置 SepLLM (n=64) 和 SepLLM (n=128) 的实验，我们发现，在训练过程中，增加邻近词元数量 (n) 会导致训练损失曲线下降更快（图 7）。此外，使用更大 n 训练的模型在下游任务中表现出更强的性能（表 2）。这突显了邻近词元在上下文语言建模和下游任务推理中的重要作用。

Hybrid Layer Benefits. 我们发现，采用某种混合架构对训练损失和下游任务性能均有益处。例如，在对应于 SepLLM (n=64) 的实验中，仅将第一个自注意力层修改为完全注意力（记为 SepLLM (n=64,H)），训练过程和下游任务均得到了适度优化。如果将第一个和最后一个注意力层都改为完全注意力（记为 SepLLM (n=64,H/T)），这种优化则更为显著。例如，LAMBADA 困惑度从 SepLLM (n=64) 的 40.08 降至 SepLLM (n=64,H) 的 36.54，以及 SepLLM (n=64,H/T) 的 33.41。

Separators’ Role. 设置 StrmLLM (n=64) 的实验对应于 SepLLM (n=64)，但未考虑邻近词元和初始词元之外的分隔符。我们观察到 StrmLLM (n=64) 的训练损失下降速度显著减慢，并且其在各种下游任务上的性能均有所下降。这表明对应于分隔符的 KV 确实包含了其所属片段的信息，这些信息有助于预测后续词元。

我们还研究了不同架构在相同输入数据上训练所需的浮点运算次数 (FLOPs) 和注意力图比率（表示注

	GSM8K-CoT			MMLU				Overall	r.KV (%)
	flexible	strict	r.KV (%)	humanities	stem	social	other		
Vanilla	77.79	77.26	100.00	60.49	56.61	76.50	72.19	65.72	100.00
StrmLLM (n=380)	70.89	71.42	47.54	57.73	54.46	74.39	70.13	63.39	52.50
StrmLLM (n=256)	69.67	68.61	26.00	62.10	54.49	73.06	69.78	62.10	37.73
SepLLM (n=256)	77.18	77.18	47.36	57.66	56.49	76.21	72.19	64.68	44.61

Table 1. GSM8K-CoT 8 样本和 MMLU 5 样本无需训练实验的评估结果及平均 runtime KV 缓存使用量。对于 SepLLM 和 StreamingLLM，本实验保留了前三个初始词元的 KV。r.KV (%) 此处表示在 runtime 时，各方法相较于 Vanilla 方法的 KV 使用量比率。更多结果见附录 I 和表 24。

Method	ARC-c	ARC-e	LBD-ppl	LBD-acc	LogiQA	PIQA	SciQ	Atten. (%)	r.KV (%)
Vanilla	20.14	46.80	34.83	33.28	23.81	62.84	81.50	100.00	100.00
StrmLLM(n=64)	20.65	47.39	44.03	26.74	21.97	63.82	75.80	16.58	15.28
SepLLM(n=64)	19.62	46.46	40.08	28.97	26.42	63.82	80.10	25.83	25.40
SepLLM(n=128)	19.97	47.35	30.16	33.18	22.73	64.64	82.60	35.64	32.27
SepLLM(n=64,H)	20.73	48.44	36.54	30.45	25.35	64.36	80.60	32.01	31.58
SepLLM(n=64,H/T)	21.42	47.26	33.41	32.80	22.73	63.98	81.20	38.18	37.75

Table 2. 在从头开始训练设置中，下游任务的性能以及平均 runtime KV 缓存使用情况。

Arch.	StrmLLM	SepLLM				Vanilla
Setting	n=64	n=64	n=128	n=64,H	n=64,H/T	full
FLOPs (%)	70.11	71.77	72.58	72.83	73.90	100.0
Atten. (%)	6.43	17.21	22.48	24.11	31.01	100.0

Table 3. FLOPs 与注意力图比率的比较。

意力掩码下三角中‘1’的比例)。如表 10 所示，我们发现 SepLLM 能够显著减少约 30% 的浮点运算次数。在绘制了相同浮点运算次数下 SepLLM 与 Vanilla 的损失比率后（见图 7(b)），我们观察到 SepLLM 的损失低于 Vanilla。这表明我们的 SepLLM 架构在训练过程中从数据集中提取有用信息的能力至少与 Vanilla 相当。此外，每次迭代的详细挂钟时间及挂钟时间加速情况见附录 C 和图 1。

4.4. Post-Training

由于从头开始训练耗时较长，我们也使用 Pythia 官方发布的 93000 步 Pythia-1.4B-deduped 检查点进行后训练实验（详见第 4.1 节）。图 8 展示了后训练的损失曲线，其中 SepLLM (n=64, 较大学习率) 表示我们采用了与原始 Pythia-1.4B-deduped 从第 0 步开始完全相同的余弦学习率调度器（包括从 0

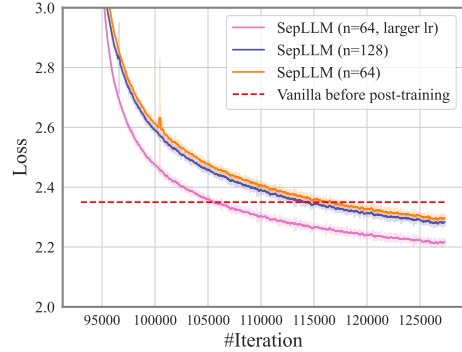


Figure 6. 后训练设置下的训练损失曲线。

开始的热身过程)。SepLLM (n=64) 和 SepLLM (n=128) 则使用了从第 93000 步开始继续衰减的余弦学习率调度器。从图 8 可以明显看出，增加 n 并适当提高学习率都有助于降低损失。此外，这也说明 SepLLM 能够通过后训练，快速地将一个全注意力 LLM 检查点转变为符合 SepLLM 架构嵌入分布要求的模型。

4.5. Streaming Applications

SepLLM 同样能很好地适应流式应用，其中可能发生无限长度的交互。在此，我们遵循 StreamingLLM (Xiao et al., 2024b)，使用我们的

PG19	1M	1.5M	2M	2.5M	3M	3.5M	4M
StrmLLM	39.5	38.2	38.3	37.6	36.4	35.8	36.1
SepLLM (s=32)	37.7	36.6	36.6	36.0	34.9	34.2	34.5
SepLLM (s=64)	37.1	36.0	36.1	35.4	34.3	33.7	33.9

Table 4. 在 PG19 测试集 (Rae et al., 2020) 上的困惑度比较。为公平评估，我们为 StreamingLLM 和 c 为 324，初始缓存容量 a 为 4。w=224，s=32/64 用于 SepLLM。

Length	Methods	c	r.KV	ppl	time (s)
20K	Vanilla	20K	10K	302.6	523.8
	StrmLLM	800	800	31.5	341.2
	SepLLM	800	562	28.3	325.8
64K	Vanilla	64K	32K	1090.8	3380.6
	StrmLLM	800	800	37.9	1096.0
	SepLLM	800	562	33.4	1049.7

Table 5. 在 PG19 测试集 (Rae et al., 2020) 上的平均困惑度和运行时间比较。r.KV 表示生成过程中平均 runtime KV 缓存使用量。

Tailored Streaming Design 在常用的 PG19 数据集 (Rae et al., 2020) 上验证无限长度交互的场景，该数据集包含 100 部篇幅较长的文学作品。结果如表 11 所示。我们可以观察到，在相同的 KV 缓存容量 c 下，通过 SepLLM 预测下一个词元的平均困惑度在输入长度从 1M 到 4M 的范围内始终低于 streamingLLM (Xiao et al., 2024b)。这再次验证了对应于分隔符的 KV 压缩片段信息的能力及其对预测下一个词元概率分布的影响。

我们还在基于 LLaMA-3-8B (Dubey 等人, 2024) 的 PG19 (Rae et al., 2020) 测试集上测试了 Vanilla、StreamingLLM 和我们的 SepLLM 的端到端推理时间。基于上述设置，我们使用这些大语言模型生成 20K 和 64K 个词元，以评估它们的总推理时间（挂钟时间）、平均困惑度以及平均运行时 KV 缓存使用量。对于 SepLLM 和 StreamingLLM，最大整体 KV 缓存容量设置为 800（即 c=800），初始缓存容量设置为 4（即 a=4）。对于 SepLLM，我们额外设置 s=64 和 w=256。结果如表 12 所示。

这些结果表明，在相同的最大 KV 缓存容量 c 下，我们的 SepLLM 能够以更短的挂钟时间和更低的平均运行时 KV 使用量实现更低的困惑度，尤其是对

s	5K	10K	15K	20K	r.KV
32	13.11	11.31	8.74	8.79	292
48	13.03	11.26	8.70	8.76	300
64	13.01	11.17	8.67	8.72	308

Table 6. SepLLM 在不同分隔符缓存容量 (s) 下在 WikiText (Merity 等人, 2017) 上的困惑度与平均 runtime KV 缓存使用情况，其中 a=4，w=224，c=324。

Method	w	c	r.KV	5K	10K	15K	20K
StrmLLM	320	324	324	13.18	11.51	8.85	8.91
	512	516	516	12.87	11.37	8.74	8.78
	796	800	800	11.96	11.01	8.67	8.72
SepLLM	224	324	308	13.01	11.17	8.67	8.72
	320	516	452	12.91	11.26	8.67	8.72
	512	800	690	12.09	11.03	8.56	8.62

Table 7. 在不同输入长度上的平均下游性能（困惑度）以及使用不同 c, w 在 WikiText 上的平均 runtime KV 使用情况，其中两种方法的 a=4，且 SepLLM 的 s=64。

于更长的序列。

4.6. Ablation Study

Hyperparameters. 我们专门针对长输入应用进行了多项消融实验。这包括详细研究不同文本长度（5K 至 20K）下各种超参数的影响。关于 s 以及 (w, c) 对的实验结果分别展示在表 13 和表 14 中。结论如下。

- s: 从表 13 可知，分隔符缓存的容量影响长文本推理的困惑度，因为我们发现增加 s 会导致困惑度一定程度的降低。
- c 和 w: 如表 14 所示，c 和 w 会影响长文本流式输入场景下的平均困惑度。此外，随着它们的增加，困惑度相应降低。

Settings. 我们还进行了以下实验，以验证初始令牌和位置编码偏移对于 StreamingLLM 和 SepLLM 的有效性。实验结果如表 15 所示，讨论如下。

- Initial Tokens: 初始令牌对于建模长流式输入的上下文至关重要，无论是对于 SepLLM 还是 StreamingLLM。移除它们会对长文本的困惑度产生显著影响。这一结论与论文 (Xiao et al., 2024b) 一致。

Method	initial	shift	5K	10K	15K	20K	r.KV
StrmLLM	✓	✓	13.2	11.5	8.9	8.9	324
StrmLLM	✗	✓	14.6	13.2	10.8	10.9	324
StrmLLM	✓	✗	425.5	513.1	509.5	506.8	324
StrmLLM	✗	✗	409.4	540.5	527.5	558.2	324
SepLLM	✓	✓	13.1	11.3	8.7	8.8	292
SepLLM	✗	✓	14.9	14.3	12.4	12.5	290
SepLLM	✓	✗	192.7	214.6	175.0	174.4	292
SepLLM	✗	✗	226.4	264.7	227.5	228.8	290

Table 8. SepLLM 和 StreamingLLM 在 WikiText (Merity 等人, 2017) 上测试的困惑度与平均 runtime KV 缓存使用率。 $c=324, a=0/4$ (两种方法相同)。 $s=32, w=224$ (SepLLM)

- Positional Encoding’s Shifting. 遵循 StreamingLLM, 我们对流式应用进行位置偏移, 即我们关注缓存内的位置而非原始文本中的位置。表 15 显示, 这种偏移起着关键作用, 因为移除它会显著增加困惑度 (StreamingLLM 从大约 13 增加到超过 400)。值得注意的是, SepLLM 即使不采用这种偏移, 困惑度也只增加到大约 200, 远低于 StreamingLLM。这进一步强调了分隔符在预测令牌稳定性方面的作用。

Separator Choices. 我们还对分隔符的选择进行了一系列消融研究。对于 SepLLM, 无论是 Fundamental Design 还是 Tailored Streaming Design, 分隔符类型的选择都作为一种超参数。鉴于常见分隔符的类型有限 (例如, 我们在此采用的 [“,” , “,” , “?” , “!” , “;” , “:” , “ ” , “\t” , “\n”]), 我们建议包含所有常用类型。关于分隔符消融研究的详细讨论见附录 G。

4.7. Comparison with More Baselines and Variants

Naive Baseline. 我们比较了 SepLLM 与更多基线及变体以验证其有效性。表 16 所示结果基于一个简单基线, 其中 H_s 个注意力头被配置为滑动窗口注意力, 而其余头使用全注意力。我们使用 MMLU 作为基准来衡量推理能力的差异, 并确保所有方法的运行时 KV 使用尽可能保持一致。可以观察到, 这种简单的按头基线表现不佳 (无论保留多少个全注意力头), 这是由于头之间的异质性所致。相比之下, 在相同的 KV 使用量下, SepLLM 在各个知识领域

均取得了与 Vanilla 非常接近的性能。

State-of-the-Art Baselines. 此外, 我们将 SepLLM 与多种最先进的基线方法进行比较, 这证明了 SepLLM 的有效性和简洁性。详细结果请参阅附录 E。

Fixed-Interval Variant. 进一步地, 为展示分隔符对其所划分片段内信息的压缩与汇总效果, 我们提出了一个固定间隔变体 FixLLM。在 FixLLM 中, 除了“初始令牌”和“邻近令牌”部分外, 模型不再关注分隔符令牌, 而是以固定间隔关注一个令牌。我们发现 SepLLM 显著优于 FixLLM, 这表明 SepLLM 中分隔符的压缩与汇总能力无法被固定间隔令牌所替代。详细结果与讨论请参见附录 I。H

4.8. Generalization and Information Retrieval

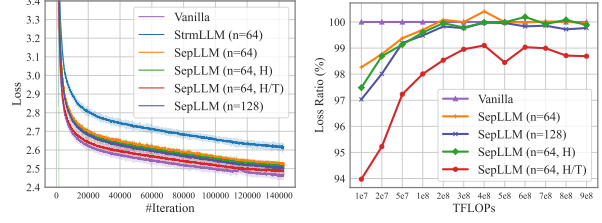
为验证 SepLLM 的泛化能力, 我们将其适配到不同架构和规模的模型上。附录 D 中的结果与讨论可以验证我们提出的 SepLLM 的泛化能力。具体而言, 我们将 SepLLM 适配到不同的骨干模型, 包括 Pythia-6.9B、Pythia-12B (Biderman et al., 2023)、Llama-3-8B-Base/Instruct (Dubey 等人, 2024) 和 Falcon-40B (Almazrouei et al., 2023)。此外, 我们还进行了 Needle-in-a-Haystack 实验, 进一步证明了分隔符令牌对片段信息的压缩能力。如附录 F 所示, SepLLM 在大多数场景下都能检索到目标信息。相比之下, StreamingLLM (Xiao et al., 2024b) 无法完成此任务。另外, 我们在附录 E 中提供了 SepLLM 与更多基线模型在数学推理和逻辑分析能力方面的比较。此外, 附录 H 和 I 从实现逻辑的角度详细讨论了分隔符的效果与功能。

5. Universal Approximation

在本节中, 我们提出关于基于编码器的 SepLLM 的通用逼近能力的理论结果。详情请参阅附录 J 和 K。

令 $\mathcal{T}_{\text{Sep}}^{H,d_h,d_f}$ 表示 SepLLM 的类别, 其中 H 、 d_h 和 d_f 分别表示注意力头的数量、注意力层的隐藏维度和前馈层的隐藏维度。记 $\mathcal{F}$ 为连续函数 $f: [0, 1]^{d \times n} \rightarrow \mathbb{R}^{d \times n}$ 的类别, 其中 d 和 n 分别表示输入令牌的维

H_s/H	MMLU					r.KV (%)	n
	humanity	social	stem	other	overall		
20/32	23.93	23.07	24.42	23.53	23.76	44.74	80
24/32	24.23	23.30	26.36	23.37	24.31	47.71	208
28/32	25.66	27.29	26.31	23.69	25.81	45.13	256
30/32	27.29	25.09	27.78	38.11	29.31	45.42	288
SepLLM	57.66	76.21	56.49	72.19	64.68	44.61	256
Vanilla	60.49	76.50	56.61	72.19	65.72	100.00	ALL



(a) 损失相对于步数 (b) 损失比率相对于 FLOPs

Figure 7. 从头开始训练的损失曲线。7(b) 显示了不同方法的损失值与 Vanilla 方法损失值相对于 FLOPs 的比率。

Table 9. 与一种朴素基线方法的比较，其中 H_s 个注意力头（共 H 个）被设置为滑动窗口注意力，其余头使用完全注意力。 n 表示相邻令牌的数量（即滑动窗口大小）。r.KV (%) 表示在 runtime 时，相应方法的 KV 使用量与原始方法 (Vanilla) 的比值。

度和序列长度。对于任意 $p \geq 1$ ，我们使用 ℓ_p 距离来衡量两个连续函数 $f_1, f_2 \in \mathcal{F}$ 之间的差异，其定义为 $\left(\int_{[0,1]^{d \times n}} \|f_1(\mathbf{X}) - f_2(\mathbf{X})\|_p^p d\mathbf{X}\right)^{1/p}$ 。以下定理表明，所提出的 SepLLM 对任意序列到序列的连续函数具有通用逼近能力。完整的证明和推导过程，请参阅附录 J 和 K。

定理 5.1. 给定 $p > 1$ 和 $n > 2$ ，对于任意 $\epsilon > 0$ 和 $f \in \mathcal{F}$ ，存在一个 SepLLM $g \in \mathcal{T}_{\text{Sep}}^{2,1,4}$ ，使得 $d_p(f, g) < \epsilon$ 。

SepLLM (n % %StrmLLM (n=256) 设置对应于从 SepLLM (n=256) 设置中移除所有分隔符的 KV，但邻近 (Neighboring) 和初始 (Initial) 标记中的那些除外。我们观察到 StrmLLM (n=256) 在数学分析和多学科知识推理能力上均出现明显下降。StrmLLM (n=256) 对于 GSM8K 和 MMLU 任务分别仅使用了 26.00% 和 37.73% 的 KV，这低于 SepLLM (n=256) (分别为 47.36% 和 44.61%)。因此，我们将 StrmLLM 的 n 增加到 380，使 GSM8K 上保留的 KV 与 SepLLM (n=256) 持平 (约 47%)，而在 MMLU 任务上，StrmLLM (n=380) 保留了 52.5% 的 KV，显著高于 SepLLM (n=256)。这带来了相比 StrmLLM (n=256) 的性能提升。然而，它仍然低于完全注意力的 Llama-3 和 SepLLM (n=256)。这表明分隔符的 KV 确实封装了其各自段落中包含的信息，移除它们会显著影响 Transformer 的理解和推理能力。

5.1. Training from Scratch

我们训练了原始的 Pythia-160m-deduped 模型，以及采用 SepLLM (及 StreamingLLM) 架构修改后的 Pythia-160m-deduped 模型，在 Pile 数据集上进行了 143,000 步训练，全局批次大小为 1024 (训练总共涉及约 300B 个词元)。所有训练配置均与 Pythia (Biderman et al., 2023) 保持一致 (参见第 4.1 节)。并且遵循 Pythia (Biderman et al., 2023)，我们在以下下游任务上进行了测试：ARC-Challenge 和 ARC-Easy (Clark 等人, 2018)、LAMBADA (Paperno 等人, 2016) (针对困惑度和准确率)、LogiQA (Liu 等人, 2021)、PIQA (Bisk et al., 2020)、SciQA (Welbl et al., 2017)。根据图 7 中描绘的损失曲线以及表 2 中的下游性能，我们得出以下分析。

Neighboring Token Benefits. 基于设置 SepLLM (n=64) 和 SepLLM (n=128) 的实验，我们发现，在训练过程中，增加邻近词元数量 (n) 会导致训练损失曲线下更快 (图 7)。此外，使用更大 n 训练的模型在下游任务中表现出更强的性能 (表 2)。这突显了邻近词元在上下文语言建模和下游任务推理中的重要作用。

Hybrid Layer Benefits. 我们发现，采用某种混合架构对训练损失和下游任务性能均有益处。例如，在对应于 SepLLM (n=64) 的实验中，仅将第一个自注意力层修改为完全注意力 (记为 SepLLM (n=64,H))，训练过程和下游任务均得到了适度优化。如果将第一个和最后一个注意力层都改为完全注意力 (记为 SepLLM (n=64,H/T))，这种优化则更为显著。例

Arch.	StrmLLM	SepLLM				Vanilla
Setting	n=64	n=64	n=128	n=64,H	n=64,H/T	full
FLOPs (%)	70.11	71.77	72.58	72.83	73.90	100.0
Atten. (%)	6.43	17.21	22.48	24.11	31.01	100.0

Table 10. FLOPs 与注意力图比率的比较。

如, LAMBADA 困惑度从 SepLLM (n=64) 的 40.08 降至 SepLLM (n=64,H) 的 36.54, 以及 SepLLM (n=64,H/T) 的 33.41。

Separators' Role. 设置 StrmLLM (n=64) 的实验对应于 SepLLM (n=64), 但未考虑邻近词元和初始词元之外的分隔符。我们观察到 StrmLLM (n=64) 的训练损失下降速度显著减慢, 并且其在各种下游任务上的性能均有所下降。这表明对应于分隔符的 KV 确实包含了其所属片段的信息, 这些信息有助于预测后续词元。

我们还研究了不同架构在相同输入数据上训练所需的浮点运算次数 (FLOPs) 和注意力图比率 (表示注意力掩码下三角中 '1' 的比例)。如表 10 所示, 我们发现 SepLLM 能够显著减少约 30% 的浮点运算次数。在绘制了相同浮点运算次数下 SepLLM 与 Vanilla 的损失比率后 (见图 7(b)), 我们观察到 SepLLM 的损失低于 Vanilla。这表明我们的 SepLLM 架构在训练过程中从数据集中提取有用信息的能力至少与 Vanilla 相当。此外, 每次迭代的详细挂钟时间及挂钟时间加速情况见附录 C 和图 1。

5.2. Post-Training

由于从头开始训练耗时较长, 我们也使用 Pythia 官方发布的 93000 步 Pythia-1.4B-deduped 检查点进行后训练实验 (详见第 4.1 节)。图 8 展示了后训练的损失曲线, 其中 SepLLM (n=64, 较大学习率) 表示我们采用了与原始 Pythia-1.4B-deduped 从第 0 步开始完全相同的余弦学习率调度器 (包括从 0 开始的热身过程)。SepLLM (n=64) 和 SepLLM (n=128) 则使用了从第 93000 步开始继续衰减的余弦学习率调度器。从图 8 可以明显看出, 增加 n 并适当提高学习率都有助于降低损失。此外, 这也说

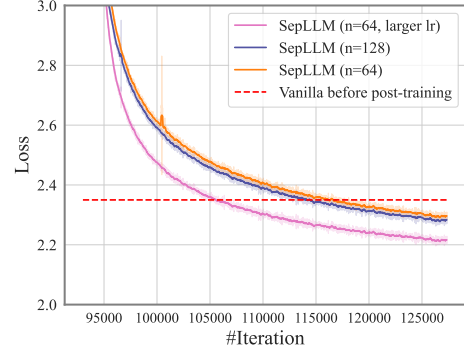


Figure 8. 后训练设置下的训练损失曲线。

PG19	1M	1.5M	2M	2.5M	3M	3.5M	4M
StrmLLM	39.5	38.2	38.3	37.6	36.4	35.8	36.1
SepLLM (s=32)	37.7	36.6	36.6	36.0	34.9	34.2	34.5
SepLLM (s=64)	37.1	36.0	36.1	35.4	34.3	33.7	33.9

Table 11. 在 PG19 测试集 (Rae et al., 2020) 上的困惑度比较。为公平评估, 我们为 StreamingLLM 和 c 为 324, 初始缓存容量 a 为 4。w=224, s=32/64 用于 SepLLM。

明 SepLLM 能够通过后训练, 快速地将一个全注意力 LLM 检查点转变为符合 SepLLM 架构嵌入分布要求的模型。

5.3. Streaming Applications

SepLLM 同样能很好地适应流式应用, 其中可能发生无限长度的交互。在此, 我们遵循 StreamingLLM (Xiao et al., 2024b), 使用我们的 Tailored Streaming Design 在常用的 PG19 数据集 (Rae et al., 2020) 上验证无限长度交互的场景, 该数据集包含 100 部篇幅较长的文学作品。结果如表 11 所示。我们可以观察到, 在相同的 KV 缓存容量 c 下, 通过 SepLLM 预测下一个词元的平均困惑度在输入长度从 1M 到 4M 的范围内始终低于 streamingLLM (Xiao et al., 2024b)。这再次验证了对应于分隔符的 KV 压缩片段信息的能力及其对预测下一个词元概率分布的影响。

我们还在基于 LLaMA-3-8B (Dubey 等人, 2024) 的 PG19 (Rae et al., 2020) 测试集上测试了 Vanilla、StreamingLLM 和我们的 SepLLM 的端到端推理时间。基于上述设置, 我们使用这些大语言模型生成 20K 和 64K 个词元, 以评估它们的总推理时间 (挂钟时间)、平均困惑度以及平均运行时 KV 缓存使

Length	Methods	c	r.KV	ppl	time (s)	Method	w	c	r.KV	5K	10K	15K	20K
20K	Vanilla	20K	10K	302.6	523.8	StrmLLM	320	324	324	13.18	11.51	8.85	8.91
	StrmLLM	800	800	31.5	341.2		512	516	516	12.87	11.37	8.74	8.78
	SepLLM	800	562	28.3	325.8		796	800	800	11.96	11.01	8.67	8.72
64K	Vanilla	64K	32K	1090.8	3380.6	SepLLM	224	324	308	13.01	11.17	8.67	8.72
	StrmLLM	800	800	37.9	1096.0		320	516	452	12.91	11.26	8.67	8.72
	SepLLM	800	562	33.4	1049.7		512	800	690	12.09	11.03	8.56	8.62

Table 12. 在 PG19 测试集 (Rae et al., 2020) 上的平均困惑度和运行时间比较。r.KV 表示生成过程中平均 runtime KV 缓存使用量。

s	5K	10K	15K	20K	r.KV
32	13.11	11.31	8.74	8.79	292
48	13.03	11.26	8.70	8.76	300
64	13.01	11.17	8.67	8.72	308

Table 13. SepLLM 在不同分隔符缓存容量 (s) 下在 WikiText (Merity 等人, 2017) 上的困惑度与平均 runtime KV 缓存使用情况, 其中 a=4, w=224, c=324。

用量。对于 SepLLM 和 StreamingLLM, 最大整体 KV 缓存容量设置为 800 (即 c=800), 初始缓存容量设置为 4 (即 a=4)。对于 SepLLM, 我们额外设置 s=64 和 w=256。结果如表 12 所示。

这些结果表明, 在相同的最大 KV 缓存容量 c 下, 我们的 SepLLM 能够以更短的挂钟时间和更低的平均运行时 KV 使用量实现更低的困惑度, 尤其是对于更长的序列。

5.4. Ablation Study

Hyperparameters. 我们专门针对长输入应用进行了多项消融实验。这包括详细研究不同文本长度(5K 至 20K) 下各种超参数的影响。关于 s 以及 (w,c) 对的实验结果分别展示在表 13 和表 14 中。结论如下。

- s: 从表 13 可知, 分隔符缓存的容量影响长文本推理的困惑度, 因为我们发现增加 s 会导致困惑度一定程度的降低。
- c 和 w: 如表 14 所示, c 和 w 会影响长文本流式输入场景下的平均困惑度。此外, 随着它们的增加, 困惑度相应降低。

Table 14. 在不同输入长度上的平均下游性能 (困惑度) 以及使用不同 c,w 在 WikiText 上的平均 runtime KV 使用情况, 其中两种方法的 a=4, 且 SepLLM 的 s=64。

Settings. 我们还进行了以下实验, 以验证初始令牌和位置编码偏移对于 StreamingLLM 和 SepLLM 的有效性。实验结果如表 15 所示, 讨论如下。

- Initial Tokens: 初始令牌对于建模长流式输入的上下文至关重要, 无论是对于 SepLLM 还是 StreamingLLM。移除它们会对长文本的困惑度产生显著影响。这一结论与论文 (Xiao et al., 2024b) 一致。
- Positional Encoding’s Shifting. 遵循 StreamingLLM, 我们对流式应用进行位置偏移, 即我们关注缓存内的位置而非原始文本中的位置。表 15 显示, 这种偏移起着关键作用, 因为移除它会显著增加困惑度 (StreamingLLM 从大约 13 增加到超过 400)。值得注意的是, SepLLM 即使不采用这种偏移, 困惑度也只增加到大约 200, 远低于 StreamingLLM。这进一步强调了分隔符在预测令牌稳定性方面的作用。

Separator Choices. 我们还对分隔符的选择进行了一系列消融研究。对于 SepLLM, 无论是 Fundamental Design 还是 Tailored Streaming Design, 分隔符类型的选择都作为一种超参数。鉴于常见分隔符的类型有限 (例如, 我们在此采用的 [“, “,” “?”, “!”, “.”, “:”, “ ”, “\t”, “\n”]), 我们建议包含所有常用类型。关于分隔符消融研究的详细讨论见附录 G。

5.5. Comparison with More Baselines and Variants

Naive Baseline. 我们比较了 SepLLM 与更多基线及变体以验证其有效性。表 16 所示结果基于一个简

Method	initial	shift	5K	10K	15K	20K	r.KV
StrmLLM	✓	✓	13.2	11.5	8.9	8.9	324
StrmLLM	✗	✓	14.6	13.2	10.8	10.9	324
StrmLLM	✓	✗	425.5	513.1	509.5	506.8	324
StrmLLM	✗	✗	409.4	540.5	527.5	558.2	324
SepLLM	✓	✓	13.1	11.3	8.7	8.8	292
SepLLM	✗	✓	14.9	14.3	12.4	12.5	290
SepLLM	✓	✗	192.7	214.6	175.0	174.4	292
SepLLM	✗	✗	226.4	264.7	227.5	228.8	290

Table 15. SepLLM 和 StreamingLLM 在 WikiText (Merity 等人, 2017) 上测试的困惑度与平均 runtime KV 缓存使用率。c=324, a=0/4 (两种方法相同)。s=32, w=224 (SepLLM)

单基线, 其中 H_s 个注意力头被配置为滑动窗口注意力, 而其余头使用全注意力。我们使用 MMLU 作为基准来衡量推理能力的差异, 并确保所有方法的运行时 KV 使用尽可能保持一致。可以观察到, 这种简单的按头基线表现不佳 (无论保留多少个全注意力头), 这是由于头之间的异质性所致。相比之下, 在相同的 KV 使用量下, SepLLM 在各个知识领域均取得了与 Vanilla 非常接近的性能。

State-of-the-Art Baselines. 此外, 我们将 SepLLM 与多种最先进的基线方法进行比较, 这证明了 SepLLM 的有效性和简洁性。详细结果请参阅附录 E。

Fixed-Interval Variant. 进一步地, 为展示分隔符对其所划分片段内信息的压缩与汇总效果, 我们提出了一个固定间隔变体 FixLLM。在 FixLLM 中, 除了“初始令牌”和“邻近令牌”部分外, 模型不再关注分隔符令牌, 而是以固定间隔关注一个令牌。我们发现 SepLLM 显著优于 FixLLM, 这表明 SepLLM 中分隔符的压缩与汇总能力无法被固定间隔令牌所替代。详细结果与讨论请参见附录 I。H

5.6. Generalization and Information Retrieval

为验证 SepLLM 的泛化能力, 我们将其适配到不同架构和规模的模型上。附录 D 中的结果与讨论可以验证我们提出的 SepLLM 的泛化能力。具体而言, 我们将 SepLLM 适配到不同的骨干模型, 包括

H_s/H	MMLU					r.KV (%)	n
	humanity	social	stem	other	overall		
20/32	23.93	23.07	24.42	23.53	23.76	44.74	80
24/32	24.23	23.30	26.36	23.37	24.31	47.71	208
28/32	25.66	27.29	26.31	23.69	25.81	45.13	256
30/32	27.29	25.09	27.78	38.11	29.31	45.42	288
SepLLM	57.66	76.21	56.49	72.19	64.68	44.61	256
Vanilla	60.49	76.50	56.61	72.19	65.72	100.00	ALL

Table 16. 与一种朴素基线方法的比较, 其中 H_s 个注意力头 (共 H 个) 被设置为滑动窗口注意力, 其余头使用完全注意力。n 表示相邻令牌的数量 (即滑动窗口大小)。r.KV (%) 表示在 runtime 时, 相应方法的 KV 使用量与原始方法 (Vanilla) 的比值。

Pythia-6.9B、Pythia-12B (Biderman et al., 2023)、Llama-3-8B-Base/Instruct (Dubey 等人, 2024) 和 Falcon-40B (Almazrouei et al., 2023)。此外, 我们还进行了 Needle-in-a-Haystack 实验, 进一步证明了分隔符令牌对片段信息的压缩能力。如附录 F 所示, SepLLM 在大多数场景下都能检索到目标信息。相比之下, StreamingLLM (Xiao et al., 2024b) 无法完成此任务。另外, 我们在附录 E 中提供了 SepLLM 与更多基线模型在数学推理和逻辑分析能力方面的比较。此外, 附录 H 和 I 从实现逻辑的角度详细讨论了分隔符的效果与功能。

6. Universal Approximation

在本节中, 我们提出关于基于编码器的 SepLLM 的通用逼近能力的理论结果。详情请参阅附录 J 和 K。

令 $\mathcal{T}_{\text{Sep}}^{H, d_h, d_f}$ 表示 SepLLM 的类别, 其中 H 、 d_h 和 d_f 分别表示注意力头的数量、注意力层的隐藏维度和前馈层的隐藏维度。记 $\mathcal{F}$ 为连续函数 $f: [0, 1]^{d \times n} \rightarrow \mathbb{R}^{d \times n}$ 的类别, 其中 d 和 n 分别表示输入令牌的维度和序列长度。对于任意 $p \geq 1$, 我们使用 ℓ_p 距离来衡量两个连续函数 $f_1, f_2 \in \mathcal{F}$ 之间的差异, 其定义为 $\left(\int_{[0, 1]^{d \times n}} \|f_1(\mathbf{X}) - f_2(\mathbf{X})\|_p^p d\mathbf{X} \right)^{1/p}$ 。以下定理表明, 所提出的 SepLLM 对任意序列到序列的连续函数具有通用逼近能力。完整的证明和推导过程, 请参阅附录 J 和 K。

定理 6.1. 给定 $p > 1$ 和 $n > 2$, 对于任意 $\epsilon > 0$

和 $f \in \mathcal{F}$, 存在一个 SepLLM $g \in \mathcal{T}_{\text{Sep}}^{2,1,4}$, 使得 $d_p(f, g) < \epsilon$ 。

7. Native Sparse Attention and Language Modeling

SepLLM 本质上基于自然语言的语义自然分布对稀疏性进行建模。在预训练阶段, SepLLM 有意将片段信息压缩到用于划分该片段的分隔符中。这种方法与自然语言的语义分布高度契合, 因为分隔符本身提供了对当前片段的划分与总结。被分隔出来的片段在语义上具有内在的连贯性, 构成自包含的语义单元。因此, we can view SepLLM as a native sparse attention mechanism inherent to natural language itself. 此外, SepLLM 适合替代 StreamingLLM (仅静态关注重要位置且过于稀疏) 作为大语言模型中稀疏注意力机制的基础基线模型。

8. Concluding Remarks

在本文中, 我们专注于高效的神经架构修改, 以应对大语言模型中的计算与存储挑战, 尤其是在处理长输入时。从注意力图的可视化中, 我们发现某些分隔符标记持续贡献较高的注意力分数。受此启发, 我们提出了 SepLLM, 一种新的语言建模视角和稀疏注意力机制, 将注意力计算集中于初始、相邻和分隔符标记上。为实现实际运行时间的加速, 我们还实现了硬件高效的内核。我们的免训练研究表明, 这些分隔符能有效压缩片段信息, 实现高效的信息检索。与以往的免训练方法不同, SepLLM 可以融入训练阶段, 例如从头训练或后训练, 从而减少训练与推理之间的差异。在各种设置下的大量实验证明了 SepLLM 的实际有效性。

Impact Statement

本文提出的工作旨在推动机器学习领域的发展。我们的工作可能带来许多潜在的社会影响, 但我们认为无需在此特别强调。

参考文献

- Almazrouei, E., Alobeidli, H., Alshamsi, A., Cappelli, A., Cojocaru, R., Debbah, M., Goffinet, É., Hesslow, D., Launay, J., Malartic, Q., Mazzotta, D., Noune, B., Pannier, B., and Penedo, G. The Falcon Series of Open Language Models. Preprint arXiv:2311.16867, 2023.
- Beltagy, I., Peters, M., and Cohan, A. Longformer: The Long-Document Transformer. Preprint arXiv:2004.05150, 2020.
- Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O’ Brien, K., Hallahan, E., Khan, M., Purohit, S., Prashanth, U., Raff, E., Skowron, A., Sutawika, L., and Wal, O. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In International Conference on Machine Learning, 2023.
- Bisk, Y., Zellers, R., Bras, R., Gao, J., and Choi, Y. PIQA: Reasoning about Physical Commonsense in Natural Language. In AAAI conference on artificial intelligence, 2020.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., 和 Amodei, D. 语言模型是小样本学习者。发表于 Neural Information Processing Systems, 2020 年。
- Chen, G., Xia, L., 和 Huang, C. Lightgcn: 用于推荐的简单图神经网络。发表于 Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining, WSDM ’25, 2025a. doi: 10.1145/3701551.3703536。

- Chen, G., Xia, L., 和 Huang, C. 用于推荐遗忘的预训练。发表于 Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '25, 2025b. doi: 10.1145/3726302.3730060.
- Chen, Y., Qian, S., Tang, H., Lai, X., Liu, Z., Han, S., 和 Jia, J. LongLoRA: 长上下文大语言模型的高效微调。发表于 International Conference on Learning Representations, 2024 年。
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., 和 Tafjord, O. 认为你已解决问答问题? 试试 ARC, AI2 推理挑战赛。预印本 arXiv:1803.05457, 2018 年。
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., 等人。训练验证器以解决数学文字问题。预印本 arXiv:2110.14168, 2021 年。
- Devlin, J., Chang, M., Lee, K., 和 Toutanova, K. BERT: 用于语言理解的深度双向 Transformer 预训练。发表于 North American Chapter of the Association for Computational Linguistics, 2019 年。
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., 和 Houtsby, N. 一张图像值 16x16 个词: 用于大规模图像识别的 Transformer。发表于 International Conference on Learning Representations, 2020 年。
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., 等人。Llama 3 模型群。预印本 arXiv:2407.21783, 2024 年。
- Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., 等人 The Pile: 一个用于语言建模的 800GB 多样化文本数据集。预印本 arXiv:2101.00027, 2020.
- Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. 模型告诉你该丢弃什么: 面向大语言模型的自适应 KV 缓存压缩。发表于 International Conference on Learning Representations, 2024.
- Geneva, N. and Zabararas, N. 用于物理系统建模的 Transformer。Neural Networks, 2022.
- He, Z., Feng, G., Luo, S., Yang, K., Wang, L., Xu, J., Zhang, Z., Yang, H., and He, D. 一石二鸟: 用于更好长度外推的双层位置编码。发表于 International Conference on Machine Learning, 2024.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. 测量大规模多任务语言理解。发表于 International Conference on Learning Representations, 2021.
- Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformer 是 RNN: 具有线性注意力的快速自回归 Transformer。发表于 International Conference on Machine Learning, 2020.
- Li, J., Shi, H., Wu, S., Zheng, C., Li, Z., Jiang, X., Xu, H., and Jia, J. QuickLLaMA: 面向大语言模型的查询感知推理加速。发表于 Proceedings of the 31st International Conference on Computational Linguistics, 2025.
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. SnapKV: 大语言模型在生成前就知道你在寻找什么。发表于 Advances in Neural Information Processing Systems, 2024.
- Liu, J., Cui, L., Liu, H., Huang, D., Wang, Y., and Zhang, Y. LogiQA: 一个用于逻辑推理机器阅读理解任务的挑战性数据集。发表于 International Joint Conferences on Artificial Intelligence, 2021.

- Merity, S., Xiong, C., Bradbury, J., and Socher, R. 指针哨兵混合模型. 发表于 International Conference on Learning Representations, 2017.
- Paperno, D., Kruszewski, G., Lazaridou, A., Pham, N., Bernardi, R., Pezzelle, S., Baroni, M., Boleda, G., and Fernández, R. LAMBADA 数据集: 需要广泛语篇上下文的词语预测任务. 发表于 Association for Computational Linguistics, 2016.
- PyTorch. Flexattention: 兼具 PyTorch 灵活性与 FlashAttention 性能, 2024. URL <https://pytorch.org/blog/flexattention/>.
- Rae, J., Potapenko, A., Jayakumar, S., Hillier, C., and Lillicrap, T. 用于长程序列建模的压缩变换器. 发表于 International Conference on Learning Representations, 2020 年.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. 探索统一文本到文本变换器的迁移学习极限. Journal of Machine Learning Research, 2020 年.
- Schlag, I., Irie, K., and Schmidhuber, J. 线性变换器是隐式的快速权重编程器. 发表于 International Conference on Machine Learning, 第 9355–9366 页. PMLR, 2021 年.
- Shi, H., Gao, J., Ren, X., Xu, H., Liang, X., Li, Z., and Kwok, J. SparseBERT: 重新思考自注意力中的重要性分析. 发表于 International Conference on Machine Learning, 2021 年.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: 具有旋转位置嵌入的增强型变换器. Neurocomputing, 568:127063, 2024 年.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A., Kaiser, L., and Polosukhin, I. 注意力机制即你所需. 发表于 Neural Information Processing Systems, 2017 年.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q., Zhou, D., et al. 思维链提示激发大语言模型中的推理能力. 发表于 Neural Information Processing Systems, 2022 年.
- Welbl, J., Liu, N., and Gardner, M. 众包多项选择科学问题. 发表于 Workshop on Noisy User-generated Text, 2017 年.
- Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., and Sun, M. InfLLM: 基于高效上下文记忆的大语言模型免训练长上下文外推. 发表于 Neural Information Processing Systems, 2024 年 a.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. 具有注意力汇聚的高效流式语言模型. 发表于 International Conference on Learning Representations, 2024 年 b.
- Yang, D., Han, X., Gao, Y., Hu, Y., Zhang, S., and Zhao, H. PyramidInfer: 面向高吞吐量大语言模型推理的金字塔 KV 缓存压缩. 预印本 arXiv:2405.12532, 2024 年.
- Yuan, J., Gao, H., Dai, D., Luo, J., Zhao, L., Zhang, Z., Xie, Z., Wei, Y. X., Wang, L., Xiao, Z., Wang, Y., Ruan, C., Zhang, M., Liang, W., and Zeng, W. 原生稀疏注意力: 硬件对齐且原生可训练的稀疏注意力. 预印本 arXiv:2502.11089, 2025 年.
- Yun, C., Chang, Y.-W., Bhojanapalli, S., Rawat, A. S., Reddi, S., and Kumar, S. O(n) 连接已足够表达: 稀疏 Transformer 的通用逼近能力. Advances in Neural Information Processing Systems, 33:13783–13794, 2020.
- Zaheer, M., Guruganesh, G., Dubey, K., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., and Ahmed, A. Big Bird: 用于更长序列的 Transformer. 载于 Neural Information Processing Systems, 2020.

Zhang, J., Zhao, Y., Saleh, M., and Liu, P. PEGASUS: 基于抽取间隙句进行预训练的抽象式摘要生成。载于 International Conference on Machine Learning, 2020。

Zhang, Y., Gao, B., Liu, T., Lu, K., Xiong, W., Dong, Y., Chang, B., Hu, J., Xiao, W., et al. PyramidKV: 基于金字塔式信息汇聚的动态 KV 缓存压缩。预印本 arXiv:2406.02069, 2024。

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H₂O: 用于大语言模型高效生成推理的重击者预言机。载于 Neural Information Processing Systems, 2023。

Zheng, C., Gao, Y., Chen, G., Shi, H., Xiong, J., Ren, X., Huang, C., Jiang, X., Li, Z., and Li, Y. 自调整 softmax。技术报告, 2025。

Zhu, Q., Duan, J., Chen, C., Liu, S., Li, X., Feng, G., Lv, X., Cao, H., Xiao, C., Zhang, X., et al. SampleAttention: 基于自适应结构化稀疏注意力的长上下文 LLM 推理近无损加速。预印本 arXiv:2406.15486, 2024。

Appendix

A. Visualization of Attention Scores

我们以 Llama-3-8B-instruct (Dubey 等人, 2024) 作为可视化模型。输入句子为“Natalia 在四月向她的 48 位朋友出售了发夹, 然后在五月她出售了数量减半的发夹。Natalia 在四月和五月总共出售了多少发夹? 答案: Natalia 在四月出售了 48 个发夹。在五月, 她出售的发夹数量是四月的一半, 因此她在五月出售了 $48/2=24$ 个发夹。所以, Natalia 在四月和五月总共出售了 $48+24=72$ 个发夹。答案是 72。”。不同注意力图的可视化结果如图 14,15,16 所示。

B. The Evolution of KV Caches

为了更好地解释流式设置中的动态设计, 我们在图 9 中展示了 KV 缓存的详细演变过程。可以看出, n 和 $Size_{run}$ 在 m_0 个标记之后都是周期函数。并且平均 KV 缓存使用量远小于最大容量 c 。

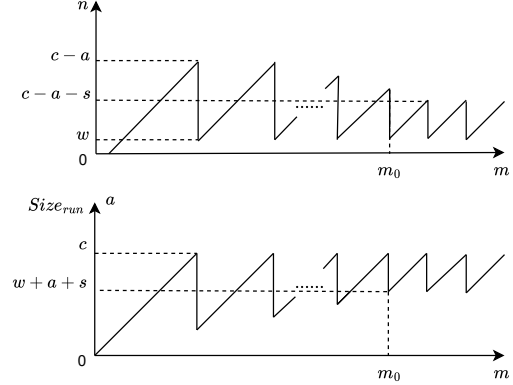


Figure 9. 流式设置中 KV 缓存的演变过程。

C. Training Acceleration

我们在表 17 中列出了每次迭代的具体墙上时间和吞吐量。加速比约为 1.53。

	Vanilla (Full Attention)	SepLLM (n=64)	SepLLM (n=128)
time per iteration (ms)	2524.45	1648.11	1653.11
samples / second	405.82	622.31	620.30

Table 17. 关于训练加速的详细信息。

我们可以看到, SepLLM(n=128) 和 SepLLM(n=64) 的速度几乎相同, 这归因于 Sep-Attention 模块出色的并行化能力。

D. The Performance of Different Models

Different Architectures. 关于不同的仅解码器模型, 我们在 Llama3 和 Pythia 骨干网络上, 于 PG19 测试数据集 (生成 64K 个标记) 上测试了我们的 SepLLM。结果如表 18 所示 (对于 SepLLM 和 StrmLLM, 均设置 $a=4$, $c=800$ 。对于 SepLLM, 设置 $s=64$, $w=256$)。

从上表可以看出, 对于规模相似的模型, 设置相似

Backbone	Arch.	c	r.KV	ppl	time(s)
Pythia-6.9B	Vanilla	64K	32K	1037.6	4160.7
	StrmLLM	800	800	15.9	1522.6
	SepLLM	800	562	15.8	1456.0
Llama-3-8B	Vanilla	64K	32K	1090.8	3380.6
	StrmLLM	800	800	37.9	1096.0
	SepLLM	800	562	33.4	1049.7

Table 18. SepLLM 适配不同架构的比较。

的 KV 保留率可以获得相似的良好性能。

Different Scales. 为了研究其不同规模上的泛化能力，我们在 Pythia-6.9B 和 Pythia-12B 骨干网络上，于 PG19 测试数据集（生成 20K 个标记）上测试了我们的 SepLLM。结果如表 19 所示。

Backbone	a	s	w	c	r.KV	ppl	time(s)
Pythia-6.9B	4	64	256	800	562	13.0	445.0
	4	64	800	1024	946	12.7	450.4
	4	64	928	1280	1138	12.7	454.4
Pythia-12B	4	64	256	800	562	12.1	577.0

Table 19. SepLLM 适配不同规模 Pythia (Biderman et al., 2023) 的比较。

与 Pythia-12B 相比，如果整个 KV 缓存容量相同 ($c=800$)，较小的模型 Pythia-6.9B 将具有更高的困惑度。因此，有必要增加 c 以实现接近 Pythia-12B ($c=800$) 的较低困惑度。另一方面，较大的模型将具有更低的困惑度，但需要更长的推理时间。

Larger Models Falcon-40B (Almazrouei et al., 2023) 是我们适配的另一个更大的架构，用于评估我们提出的 SepLLM 的可扩展性。实验结果如表 20 所示，其中我们为 SepLLM 设置 $a=4$, $s=64$, $w=512/720$, $c=800/1024$ ，为 StreamingLLM 设置 $a=4$, $c=800/1024$ 。结论与之前部分相似。

Base or Instruct. 一般而言，无论是基础模型还是指令调优模型，我们都可以将片段信息压缩到分隔符标记对应的键值对中。为了说明这一点，我们分别在 LongAlpaca 数据集 (Dubey 等人, 2024) 上对 Llama-3-8B-instruct 和 Llama-3-8B-base 模型进行

Length	Methods	c	r.KV	ppl	time (s)
20K	StrmLLM	1024	1024	8.98	1512.88
	StrmLLM	800	800	9.02	1430.59
	SepLLM	1024	906	8.92	1440.89
	SepLLM	800	690	9.00	1368.07
64K	StrmLLM	1024	1024	11.01	4844.79
	StrmLLM	800	800	11.09	4623.90
	SepLLM	1024	906	10.96	4619.63
	SepLLM	800	690	11.07	4414.72

Table 20. SepLLM 适配 Falcon-40B (Almazrouei et al., 2023) 的比较。

Backbone	Algorithm	GSM8K-CoT	r.KV (%)
8B	Base	Vanilla	54.44
	Instruct	SepLLM ft.	55.95
		Vanilla	77.26
	Instruct	SepLLM ft.	77.63

对比。

(Dubey 等人, 2024)

微调，仅分别进行 200 步和 500 步。微调后，我们采用 GSM8K-CoT (Chen 等人, 2024) 作为推理能力评估的基准。我们发现，基础模型和指令调优模型都表现出优异的性能（匹配或超越了采用原始注意力机制的原始模型）。唯一的区别在于，对于 Llama-3-8B-base，我们需要微调更多步数才能达到这样的性能。相比之下，Llama-3-8B-instruct 所需的微调步数更少。即使在无需训练的场景下，Llama-3-8B-instruct 也展现出不错的性能。这表明 Llama-3-8B-instruct 的嵌入与 SepLLM 架构所需的分布更为契合。

(Cobbe 等人, 2021)[!ht]

E. Extended Comparisons

我们使用 GSM8K-CoT (Cobbe 等人, 2021)，这是测试数学推理与逻辑分析最常用的指标，来比较 SepLLM ($a=3$, $n=256$) 与其他最先进的无训练方法，包括 H₂O (Zhang et al., 2023)、SnapKV (Li 等人, 2024) 和 PyramidKV (Zhang et al., 2024)。所有方法均配置为保留几乎相同的运行时 KV 缓存

	flexible-extract	strict-match	r.KV (%)
Vanilla	77.79	77.26	100.00
SepLLM	77.18	77.18	47.36
H2O	76.27	75.06	47.54
SnapKV	76.50	73.62	47.54
PyramidKV	75.82	72.02	47.54

Table 22. 在 GSM8K-CoT 上使用 8 样本提示的实验评估结果及平均 runtime KV 缓存使用量, 与多种基线方法的比较。

使用量。结果如表 22 所示。可以观察到, SepLLM 无需复杂的权重评估机制, 仅通过压缩片段信息便能保持强大的推理能力。

F. Needle in a Haystack

为评估我们提出的 SepLLM 的长上下文信息检索能力, 我们采用 Needle in a Haystack²作为基准, 并比较 SepLLM 与 StreamingLLM 的性能。结果如图 10、11、12、13 所示, 与 StreamingLLM 相比, SepLLM 能够获得更高的分数。该实验确实验证了 SepLLM 能够有效地将片段信息压缩到与分隔符对应的 KV 中, 因为即使与针 (needle) 中除可能存在的分隔符外的其他标记对应的 KV 被丢弃, SepLLM 仍然能够检索到该针。

Needle-in-a-Haystack Needle-in-a-Haystack
Needle-in-a-Haystack Needle-in-a-Haystack

G. Choice of Separators

在本文中, 我们将分隔符的选择视为一个超参数。由于在大多数语言中常用分隔符的数量通常有限, 我们建议尽可能包含所有类型的分隔符。在表 23 中, 我们展示了基于不同分隔符选择的实验结果。我们使用 GSM8K-CoT 指标来反映这些选择导致的性能差异 (GSM8K-CoT 是评估大语言模型数学推理与逻辑分析能力的常用指标)。可以观察到, 当使用四种常见分隔符替代默认的九种时, 性能出现一定程度的下降。此外, 当仅使用 “.” 和

“?” 时, 模型的推理能力显著下降。当移除所有分隔符时, 这种性能退化变得更加明显 (即 StrmLLM(n=256))。虽然增加邻近标记数量 (n) 以确保保留的 KV 缓存总量不变可以部分缓解性能损失 (即 StrmLLM(n=380)), 但这远未将性能恢复至默认九种分隔符或四种常见分隔符所能达到的水平。

H. Discussions on Separators

我们提供以下假设和讨论, 解释为何保留与分隔符对应的键值对能够维持原始模型的性能。

Training from Scratch 在我们的训练过程中, 我们强制规定每个当前标记只能看到其前驱相邻标记、分隔符标记以及初始标记, 从而迫使模型通过自注意力机制将每个片段的信息压缩到与分隔符对应的键值对中。因此, 分隔符的隐藏嵌入在功能上类似于循环神经网络的状态空间, 尽管计算方法不同, 因为我们利用了注意力机制。此外, 由于每个片段的长度通常是有限、短小且均衡的 (He 等人, 2024), 压缩后的信息是高效的, 且不易被遗忘。

Training-Free 首先, 这些分隔符 (逗号、句号) 是极高频率的标记 (无论是在预训练文本中, 还是在语言模型生成的文本中)。因此, 在预训练过程中, 这些分隔符是词汇表中所有其他标记最常见的上下文。因此, 它们的嵌入表现出更高的相似性, 导致与其他标记相乘时产生更大的注意力值。此外, 从逻辑角度来看, 显然分隔符需要被语言模型非常频繁地生成。因此, 它们与任何其他标记之间的注意力值不能太小; 否则, 它们就不会被语言模型频繁生成。具有较大相互注意力值的标记将包含更多来自其他标记的信息。因此, 从语义角度来看, 生成一个分隔符即是对当前片段的总结, 自然地划分并概括了上下文语义。

I. Fixed-Interval Variant

稀疏性在各种类型的神经网络中普遍存在 (Shi et al., 2021; Chen 等人, 2025a; Zheng et al., 2025; Xuan et al., 2025; Chen 等人, 2025b)。为了验证自

²https://github.com/gkamradt/LLMTest_NeedleInAHaystack

	SepLLM (n=256)	SepLLM (n=256)	SepLLM (n=256)	StrmLLM (n=256)	StrmLLM (n=380)
Separators	“.” “,” “?” “!” “;” “:” “ ” “\t” “\n”	“.” “;” “?” “;”	“.” “?”	None	None
r.KV (%)	47.36	37.92	36.44	31.92	47.54
flexible-extract	77.18	76.68	70.66	68.84	71.42
strict-match	77.18	76.85	70.36	67.63	70.89

Table 23. 基于不同分隔符选择，在 GSM8K-CoT 上使用 8 样本进行的免训练实验的评估结果及平均 runtime 键值缓存使用情况。

然语言中作为自然语义划分的分隔符所引起的稀疏性以及这些分隔符的重要性，我们提出了另一种变体，即，在计算稀疏注意力时，我们不再仅关注初始标记与相邻标记之间的分隔符标记，而是以固定间隔（例如，每 8 个标记或 16 个标记）关注一个标记，同时掩码其他标记，即 FixLLM。我们基于 Llama3-8B-Instruct 主干，在无需训练的设置下，使用 GSM8K-CoT (Cobbe 等人, 2021) 和 MMLU (Hendrycks 等人, 2021)) 基准测试评估了这两种变体的数学推理能力和基于知识的推理能力。表 24 中的结果表明，与 SepLLM 相比，FixLLM 在数学逻辑推理和基于知识的推理能力上均存在显著差距。因此，SepLLM 中分隔符对其所划片段的总结与压缩效果无法被固定间隔的标记所替代。

J. Universal Approximation

在本节中，我们从理论上分析基于编码器的 SepLLM 的通用逼近能力。令 $\mathcal{T}_{\text{Sep}}^{H,d_h,d_f}$ 表示 SepLLM 的类别，其中 H 、 d_h 和 d_f 分别代表注意力层中的头数、隐藏维度和前馈层的隐藏维度。在 SepLLM 的注意力层中，令牌 i 可以关注其大小为 ℓ 的滑动窗口内的相邻令牌（即范围 $[i-\ell, i+\ell]$ 内的令牌）以及序列的所有特殊令牌。此外，我们假设在最多 s 个连续令牌中，序列中会出现一个特殊令牌。令 $\mathcal{F}$ 表示连续函数 $f: [0, 1]^{d \times n} \rightarrow \mathbb{R}^{d \times n}$ 的类别，其中 d 和 n 分别表示输入令牌的维度和序列长度。对于任意 $p \geq 1$ ，我们使用 ℓ_p 距离来度量两个连续函数 $f_1, f_2 \in \mathcal{F}$ 之间的差异，其定义为 $\left(\int_{[0,1]^{d \times n}} \|f_1(\mathbf{X}) - f_2(\mathbf{X})\|_p^p d\mathbf{X} \right)^{1/p}$ 。以下定理表明，所提出的 SepLLM 对任意序列到序列的连续函

数具有通用逼近能力。

定理 J.1. 给定 $p > 1$ 和 $n > 2$ ，对于任意 $\epsilon > 0$ 和 $f \in \mathcal{F}$ ，存在一个 SepLLM $g \in \mathcal{T}_{\text{Sep}}^{2,1,4}$ ，使得 $d_p(f, g) < \epsilon$ 。

我们在此概述证明的关键步骤，完整细节将在后文提供。该证明遵循 Yun et al. (2020) 中的方法。

Step 1: 首先，我们将区域 $[0, 1]^{d \times n}$ 划分为一组网格点 $\mathcal{G}_\delta = \{0, \delta, 2\delta, \dots, 1\}^{d \times n}$ ，其中 $[0, 1]^{d \times n}$ 中的每个点对应于由这些网格点定义的立方体。这里， $\delta > 0$ 决定了网格点的分辨率。具体而言，对于任意 $\mathbf{X} \in [0, 1]^{d \times n}$ ，存在一个网格点 $\mathbf{X}_\delta \in \mathcal{G}_\delta$ ，使得 $\mathbf{X}$ 位于立方体 $\mathbf{X}_\delta + [0, \delta]^{d \times n}$ 内。然后，我们为属于同一立方体的所有输入分配相同的函数值，这由分段常数函数 $\bar{f}$ 确定。对于任意 $\epsilon > 0$ ，存在一个足够小的 $\delta > 0$ ，使得 $d_p(f, \bar{f}) \leq \frac{\epsilon}{2}$ 。

Step 2: 我们将 SepLLM 注意力层和前馈层中的 softmax 和 ReLU 激活替换为硬最大值算子（即 $\arg \max$ ）和分段线性函数（最多三段）。我们将修改后的 SepLLM 模型类别记为 $\bar{\mathcal{T}}_{\text{Sep}}^{H,d_h,d_f}$ 。对于上述分段线性函数 $\bar{f}$ ，存在一个修改后的 SepLLM $\bar{g} \in \bar{\mathcal{T}}_{\text{Sep}}^{2,1,1}$ ，使得 $\bar{g} = \bar{f}$ 。

Step 3: 最后，我们通过标准 SepLLM $g \in \mathcal{T}_{\text{Sep}}^{2,1,4}$ 来逼近修改后的 SepLLM $\bar{g} \in \bar{\mathcal{T}}_{\text{Sep}}^{2,1,1}$ ，即我们有 $d_p(\bar{g}, g) < \frac{\epsilon}{2}$ 。这一逼近的合理性在于，当温度参数足够大时，softmax 函数可以任意接近地逼近硬最大值算子。此外，具有 ReLU 激活的前馈网络可以有效表示任何分段线性函数。

	GSM8K-CoT			MMLU					
	flexible	strict	r.KV (%)	humanities	stem	social	other	overall	r.KV (%)
Vanilla	77.79	77.26	100.00	60.49	56.61	76.50	72.19	65.72	100.00
FixLLM ($\Delta=5$, $n=256$)	70.43	70.43	45.64	55.52	54.80	72.99	69.75	62.33	50.20
FixLLM ($\Delta=4$, $n=256$)	72.71	72.33	49.08	55.92	54.39	74.36	70.81	62.91	53.32
SepLLM ($n=256$)	77.18	77.18	47.36	57.66	56.49	76.21	72.19	64.68	44.61

Table 24. 在 GSM8K-CoT 8 样本和 MMLU 5 样本上进行的无需训练实验的评估结果及平均 runtime KV 缓存使用情况。对于 SepLLM 和 FixLLM，保留了三个初始标记的 KV。 Δ 表示 FixLLM 的间隔大小， n 是保留的相邻标记 KV 的数量。r.KV (%) 表示在 runtime 处，相应方法的 KV 使用量与 Vanilla 方法相比的比率。

K. Proof for Theorem J.1

引理 K.1 (引理 5, 摘自 Yun et al. (2020)). 对于任意 $f \in \mathcal{F}$ 、 $\epsilon > 0$ 和 $p \geq 1$ ，存在一个分段常数函数 $\bar{f}$ ，使得 $d_p(f, \bar{f}) < \frac{\epsilon}{2}$ 。

为了在 SepLLM 中识别标记的位置，我们将位置编码 $\mathbf{E} \in \mathbb{R}^{d \times n}$ 纳入标记 $\mathbf{X}$ 中，即输入标记为 $\mathbf{X} + \mathbf{E}$ 。此处，为了方便理论分析，位置编码矩阵定义为

$$\mathbf{E} = [(n-1)\mathbf{1}, \mathbf{0}, \mathbf{1}, \dots, (n-2)\mathbf{1}].$$

通过这种编码，输入标记位于范围 $\mathbf{X} + \mathbf{E}$ 内，该范围取值于 $[0, n]^{d \times n}$ 。随后，输入标记通过一个量化函数 g_q 映射到网格点上，该函数可以使用具有分段线性激活函数的前馈神经网络（即修改后 SepLLM 的前馈层）来实现。

引理 K.2 (引理 6, 摘自 Yun et al. (2020)). 考虑量化映射 g_q^{ent} :

$$g_q^{\text{ent}}(t) = \begin{cases} k\delta, & \text{if } k\delta \leq t < (k+1)\delta, \quad k \in [0 : n/\delta - 1] \\ t, & \text{otherwise.} \end{cases} \quad (5)$$

存在一个修改后的 SepLLM，其包含前馈层和恒等注意力层， $g_q \in \bar{\mathcal{T}}_{\text{Sep}}^{2,1,1}$ 由 $\frac{nd}{\delta}$ 层组成，具有 $n_f = 1$ 和分段线性激活函数，使得 g_q 实现了量化映射 g_q^{ent} 。

令 $\mathbf{Z} = g_q(\mathbf{X} + \mathbf{E})$ 表示量化后的输入标记矩阵，其对应于将 $\mathbf{X} + \mathbf{E}$ 映射到立方体最左角的网格点。令 $\mathcal{G}_\delta$ 表示从 $\mathbf{X} + \mathbf{E}$ 导出的所有量化标记矩阵的集合。以下上下文映射的定义旨在从所有可能的组合中唯一地区分每个网格点（即量化标记）。

定义 K.3 (上下文映射). 对于给定的网格点集合 $\mathcal{G}_\delta \subset \mathbb{R}^{d \times n}$ ，一个上下文映射 $q : \mathcal{G}_\delta \rightarrow \mathbb{R}^n$ 满足：

- 对于任意 $\mathbf{G} \in \mathcal{G}_\delta$ ， $q(\mathbf{G})$ 中的所有条目都是互不相同的 $\bar{\mathbf{a}}$
- 对于任意满足 $\mathbf{G}, \mathbf{G}' \in \mathcal{G}_\delta$ 的 $\mathbf{G} \neq \mathbf{G}'$ ， $[q(\mathbf{G}), q(\mathbf{G}')] \in \mathbb{R}^{2n}$ 的所有元素都是互不相同的。

引理 K.4. 存在一个函数 $g_c \in \bar{\mathcal{T}}_{\text{Sep}}^{2,1,1} : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^{d \times n}$ ，它由 $\frac{n-1}{\delta^d} + \lceil \frac{s}{\ell} \rceil$ 层改进的 SepLLM（纯注意力层，前馈层为恒等映射）构成，使得 $q(\mathbf{Z}) := \mathbf{u}^\top g_c(\mathbf{Z})$ 是一个上下文映射。这里，改进的 SepLLM 是指将所有 softmax 函数替换为 hardmax 算子。

证明. 将 $\mathbf{Z}$ 的第 i 列记作 $\mathbf{Z}_i$ ，并令 $\mathbf{u} = (1, \delta^{-1}, \delta^{-2}, \dots, \delta^{-d+1})^\top$ 。根据量化映射 g_q 的定义以及位置编码矩阵 $\mathbf{E}$ 的选择，我们有

$$\begin{aligned} \mathbf{u}^\top \mathbf{Z}_1 &\in \left[(n-1) \sum_{i=0}^{d-1} \delta^{-i} : \delta : (n-1) \sum_{i=0}^{d-1} \delta^{-i} + \delta^{-d+1} - \delta \right], \\ \mathbf{u}^\top \mathbf{Z}_k &\in \left[(k-2) \sum_{i=0}^{d-1} \delta^{-i} : \delta : (k-2) \sum_{i=0}^{d-1} \delta^{-i} + \delta^{-d+1} - \delta \right], \end{aligned} \quad (6)$$

对于 $k \in [2 : n]$ 成立。我们观察到，对应于不同 k 的那些区间是互不相交的。将 $\mathbf{u}^\top \mathbf{Z}_k$ 记作 z_k ，则有

$z_2 < z_3 < \dots < z_n < z_1$ 。我们定义算子

$$\begin{aligned} & \Psi(\mathbf{Z}; b)_k \\ &= \mathbf{u}^T \mathbf{Z}_{\mathcal{A}_k} \sigma_H \left[(\mathbf{u}^T \mathbf{Z}_{\mathcal{A}_k})^T (\mathbf{u}^T \mathbf{Z}_k - b) \right] \\ &= \begin{cases} \max_{j \in \mathcal{A}_k} \mathbf{u}^T \mathbf{Z}_j & \text{if } \mathbf{u}^T \mathbf{Z}_k > b, \\ \min_{j \in \mathcal{A}_k} \mathbf{u}^T \mathbf{Z}_j & \text{if } \mathbf{u}^T \mathbf{Z}_k < b, \end{cases} \end{aligned}$$

其中 $\mathcal{A}_k$ 表示在 SepLLM 中词元 k 可以关注到的词元集合, 包括其邻近词元和特殊词元。算子 $\Psi(\mathbf{Z}; b)_k$ 可以在改进的 SepLLM 中使用一个单头注意力层 (以 `hardmax` 算子作为激活函数) 来实现。我们进一步定义以下算子, 它可以在改进的 SepLLM 中使用一个双头注意力层来实现:

$$\begin{aligned} & \Phi(\mathbf{Z}; c; b_{\min}, b_{\max})_k \\ &= \mathbf{Z} + c (\Psi(\mathbf{Z}; b_{\max})_k - \Psi(\mathbf{Z}; b_{\min})_k) \mathbf{e}_1 \\ &= \begin{cases} \mathbf{Z} + c (\max_{j \in \mathcal{A}_k} \mathbf{u}^T \mathbf{Z}_j - \min_{j \in \mathcal{A}_k} \mathbf{u}^T \mathbf{Z}_j) \mathbf{e}_1 \\ \mathbf{Z} \end{cases}. \end{aligned}$$

这里, 第一个条件在 $b_{\min} < \mathbf{u}^T \mathbf{Z}_k < b_{\max}$ 时满足, 否则适用第二个条件。对于 $k = 2$ (第二个词元), 我们应用算子 $\Phi(\mathbf{Z}; \delta^{-d}; b - \frac{\delta}{2}, b + \frac{\delta}{2})$ 共 δ^{-d} 次, 其中 b 在范围 $[0 : \delta : \delta^{-d+1} - \delta]$ 内变化。注意, 该算子仅修改第 k 个词元而保持所有其他词元不变, 因为 $\mathbf{u}^T \mathbf{Z}_k$ 位于互不相交的区间内。在此操作之后, 我们将更新后的第二个词元记作 $\tilde{\mathbf{Z}}_2$, 并且有

$$\tilde{\mathbf{Z}}_2 = \mathbf{Z}_2 + \delta^{-d}(z_1 - z_2)\mathbf{e}_1,$$

和

$$\tilde{z}_2 := \mathbf{u}^T \tilde{\mathbf{Z}}_2 = z_2 + (z_1 - z_2)\delta^{-d} > z_1.$$

类似地, 对于第三个词元 ($k = 3$), 我们应用算子 $\Phi(\mathbf{Z}; \delta^{-d}; b - \frac{\delta}{2}, b + \frac{\delta}{2})$, 其中 b 在范围 $[\sum_{i=0}^{d-1} \delta^{-i} : \delta : \sum_{i=0}^{d-1} \delta^{-i} + \delta^{-d+1} - \delta]$ 内变化。此操作仅修改第三个词元 (矩阵 $\mathbf{Z}$ 的第三列), 得到:

$$\tilde{\mathbf{Z}}_3 = \mathbf{Z}_3 + \delta^{-d}(\tilde{z}_2 - z_3)\mathbf{e}_1,$$

和

$$\tilde{z}_3 := \mathbf{u}^T \tilde{\mathbf{Z}}_3 = z_3 + (\tilde{z}_2 - z_3)\delta^{-d} > \tilde{z}_2.$$

我们对剩余词元重复此过程, 直至最后一个词元 $k = n$ 。最终, 得到的词元矩阵 $\tilde{\mathbf{Z}}$ 满足:

$$\tilde{z}_1 < \tilde{z}_2 < \dots < \tilde{z}_n,$$

其中 $\tilde{z}_k := \mathbf{u}^T \tilde{\mathbf{Z}}_k$ 。总之, 存在一个具有 $\frac{n-1}{\delta^d}$ 层的改进版 SepLLM, 能够将词元矩阵 $\mathbf{Z}$ 转换为 $\tilde{\mathbf{Z}}$ 。

现在, 我们证明从 $\mathbf{Z}$ 到 $\tilde{z}_n$ 的映射是单射。假设存在两个词元矩阵 $\mathbf{Z}, \mathbf{Z}' \in \mathcal{G}_\delta$, 使得 $\tilde{z}_n = \tilde{z}'_n$ 。通过归纳, 我们有

$$\begin{aligned} \tilde{z}_n &= z_n + \delta^{-d}(\tilde{z}_n - z_n) \\ &= \dots \\ &= z_n + \sum_{i=1}^{n-1} \delta^{-id}(z_{n-i} - z_{n+1-i}). \end{aligned}$$

如果 $z_n \neq z'_n$, 则 $|z_n - z'_n| \leq \delta^{-d+1} - \delta$ 。然而, 项 $\sum_{i=1}^{n-1} \delta^{-id}(z_{n-i} - z_{n+1-i})$ 是主导项, 无法抵消 $|z_n - z'_n|$ 。如果 $z_n = z'_n$ 但 $z_{n-1} \neq z'_{n-1}$, 我们会遇到类似的矛盾。由于求和中的每一项具有不同的 δ^{-1} 尺度, 因此 $\tilde{z}_n = \tilde{z}'_n$ 成立当且仅当 $\mathbf{Z} = \mathbf{Z}'$ 。对于当前的词元矩阵 $\tilde{\mathbf{Z}} = [\tilde{\mathbf{Z}}_1, \tilde{\mathbf{Z}}_2, \dots, \tilde{\mathbf{Z}}_n]$, 每一列的主导项位于第一行, 其中

$$(\tilde{\mathbf{Z}}_k)_1 = (\mathbf{Z}_k)_1 + \delta^{-d}(\tilde{z}_{k-1} - z_k).$$

此外, 每一列第一个元素的主导项是 $\delta^{-d}\tilde{z}_{k-1}$ 。由于 $\tilde{z}_n$ 可视为原始词元矩阵 $\mathbf{Z}$ 的恒等元, 依赖于 $\tilde{z}_n$ 的最后一列对于不同的词元矩阵是独特的。然而, 对于由 $\tilde{z}_k$ ($k \neq n$) 主导的其他列, 唯一性无法保证。为解决此问题, 我们应用一系列词元传递步骤, 将信息从最后一个词元传播到所有其他词元。核心思想依赖于最后一个词元能够在相邻词元的帮助下关注到最近的特殊词元, 这最多需要 $\lceil \frac{s}{\ell} \rceil$ 层。根据 (Yun et al., 2020) 中的第 E.2.4 节, 经过 $\lceil \frac{s}{\ell} \rceil$ 层后, $\tilde{z}_n$ 被成功复制到其他词元。因此, 更新后的词元矩阵 (记为 $\tilde{\tilde{\mathbf{Z}}}$) 的每一列都依赖于 $\tilde{z}_n$ 。此外, $\mathbf{u}^T \tilde{\tilde{\mathbf{Z}}}$ 的所有元素都是独特的, 且位于不相交的区间内。由于 $\tilde{z}_n$ 充当词元矩阵 $\mathbf{Z}$ 的恒等元, 映射 $\mathbf{u}^T \tilde{\tilde{\mathbf{Z}}}$ 满足上下文映射的所有条件。从 $\mathbf{Z}$ 到 $\tilde{\tilde{\mathbf{Z}}}$ 的映射可以通过 $\frac{n-1}{\delta^d} + \lceil \frac{s}{\ell} \rceil$ 层改进的 SepLLM 实现, 该网络仅由注意力层和恒等前馈层构成。□

引理 K.5 (Yun et al. (2020) 中的引理 8). 对于引理 K.4 中的上下文映射 g_c , 存在 $\frac{n}{\delta^{dn}}$ 层改进的 SepLLM (由前馈层和恒等注意力层组成), 记为 $g_v \in \tilde{\mathcal{T}}_{\text{Sep}}^{2,1,1}$:

$\mathbb{R}^{d \times n} \rightarrow \mathbb{R}^{d \times n}$, 使得

$$g_v(g_c(\mathbf{Z}))_k = \bar{f}(\mathbf{Z} - \mathbf{E})_k,$$

其中 k 表示词元矩阵的第 k 列。

根据以上分析, 我们得到

$$\bar{g}(\mathbf{X}) = g_v \circ g_c \circ g_q(\mathbf{X} + \mathbf{E}) = \bar{f}(\mathbf{X}),$$

其中量化映射 g_q 、上下文映射 g_c 和值映射 g_v 均由

改进的 SepLLM 实现。此处 $\bar{g}$ 表示具有

$\frac{nd}{\delta} + \frac{n-1}{\delta^d} + \lceil \frac{s}{\ell} \rceil + \frac{n}{\delta^{nd}}$ 层的改进 SepLLM。以下引理表明, 改进的 SepLLM 可以用标准 SepLLM 以任意小的误差逼近。

引理 K.6 (Yun et al. (2020) 中的引理 4). 对于任意改进的 SepLLM $\bar{g} \in \bar{\mathcal{T}}_{\text{Sep}}^{2,1,1}$ 和任意 $\epsilon > 0$, 存在一个 SepLLM $g \in \mathcal{T}_{\text{Sep}}^{2,1,4}$, 使得 $d_p(g, \bar{g}) < \epsilon$ 。

综合以上所有引理, 我们得出结论: 对于任意 $f \in \mathcal{F}$ 和 $\epsilon > 0$, 存在一个 SepLLM $g \in \mathcal{T}_{\text{Sep}}^{2,1,4}$, 使得 $d_p(f, g) < \epsilon$ 。

L. AI Assistant

我们仅使用人工智能助手（例如, GPT-3.5-Turbo）来润色和完善文章。

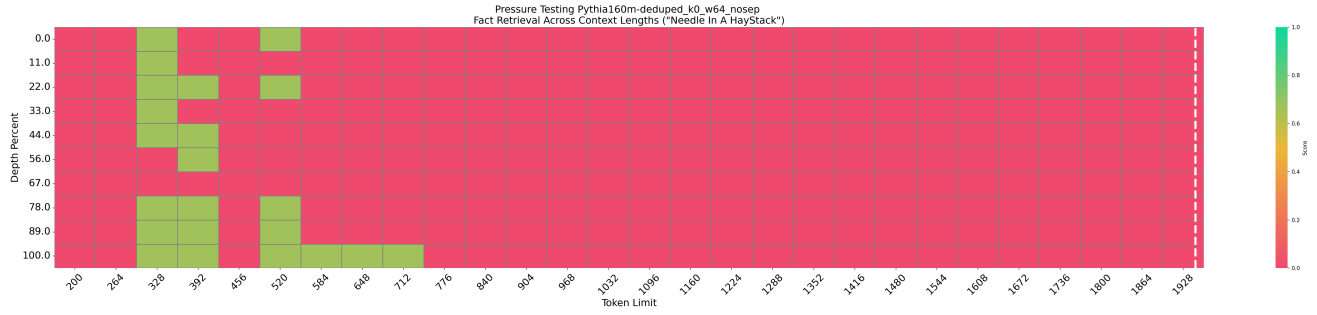


Figure 10. 基于 Pythia-160M-deduped 的 StreamingLLM (n=64) 测试结果。

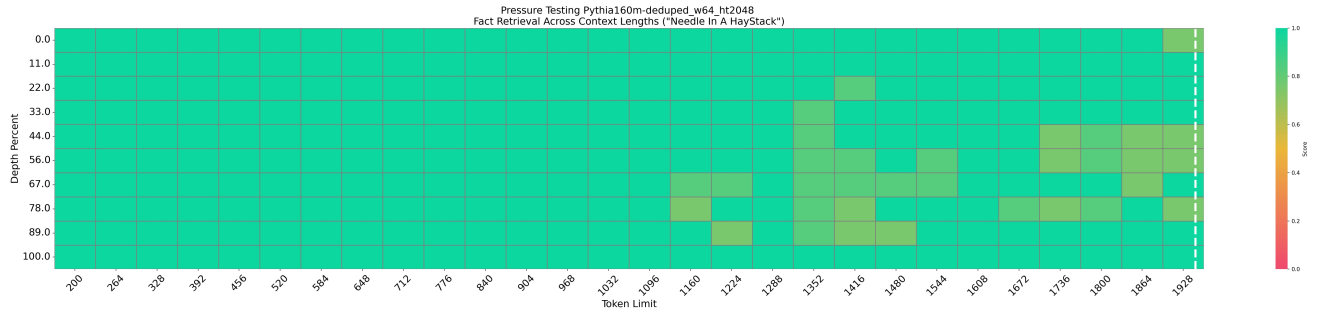


Figure 11. 基于 Pythia-160M-deduped 的我们的 SepLLM (n=64, H/T) 测试结果。

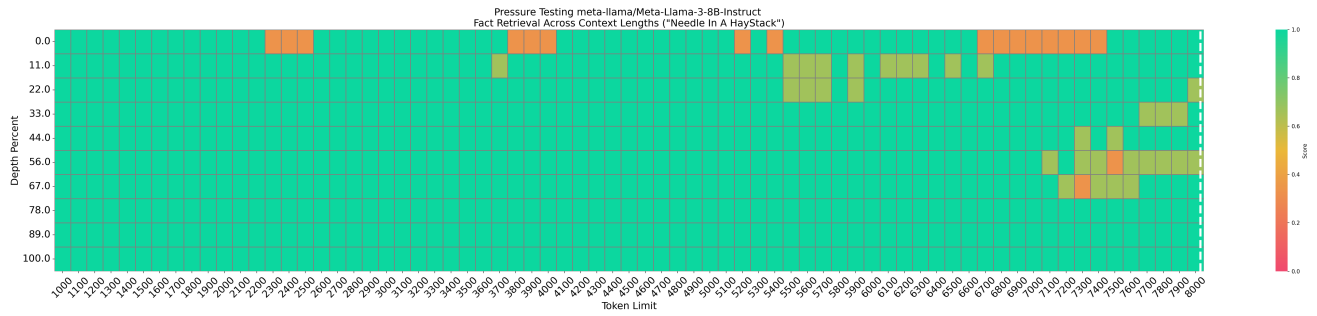


Figure 12. 基于 Llama-3-8B-instruct 的我们的 SepLLM (n=2048; 首尾各 2 层 (共 4 层): 完全注意力) 测试结果。保留前 4 个初始标记。

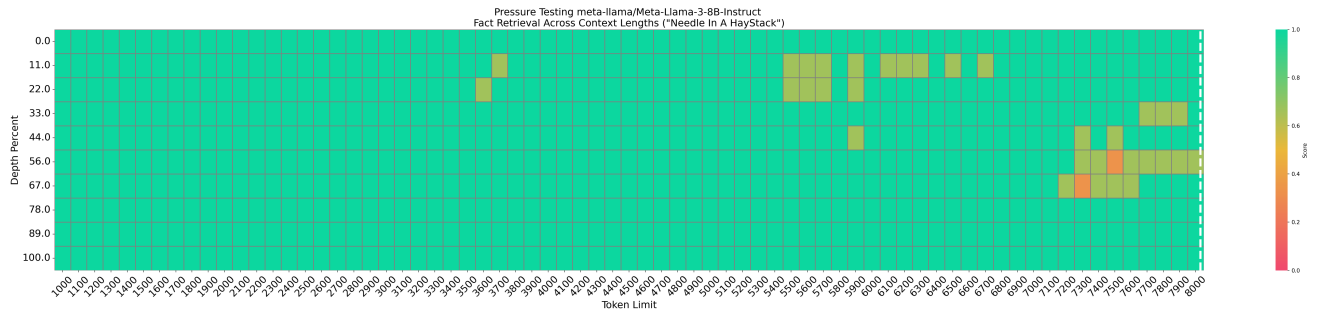


Figure 13. 基于 Llama-3-8B-instruct 的我们的 SepLLM (n=2048; 首尾各 2 层 (共 4 层): 完全注意力) 测试结果。保留前 32 个初始标记。

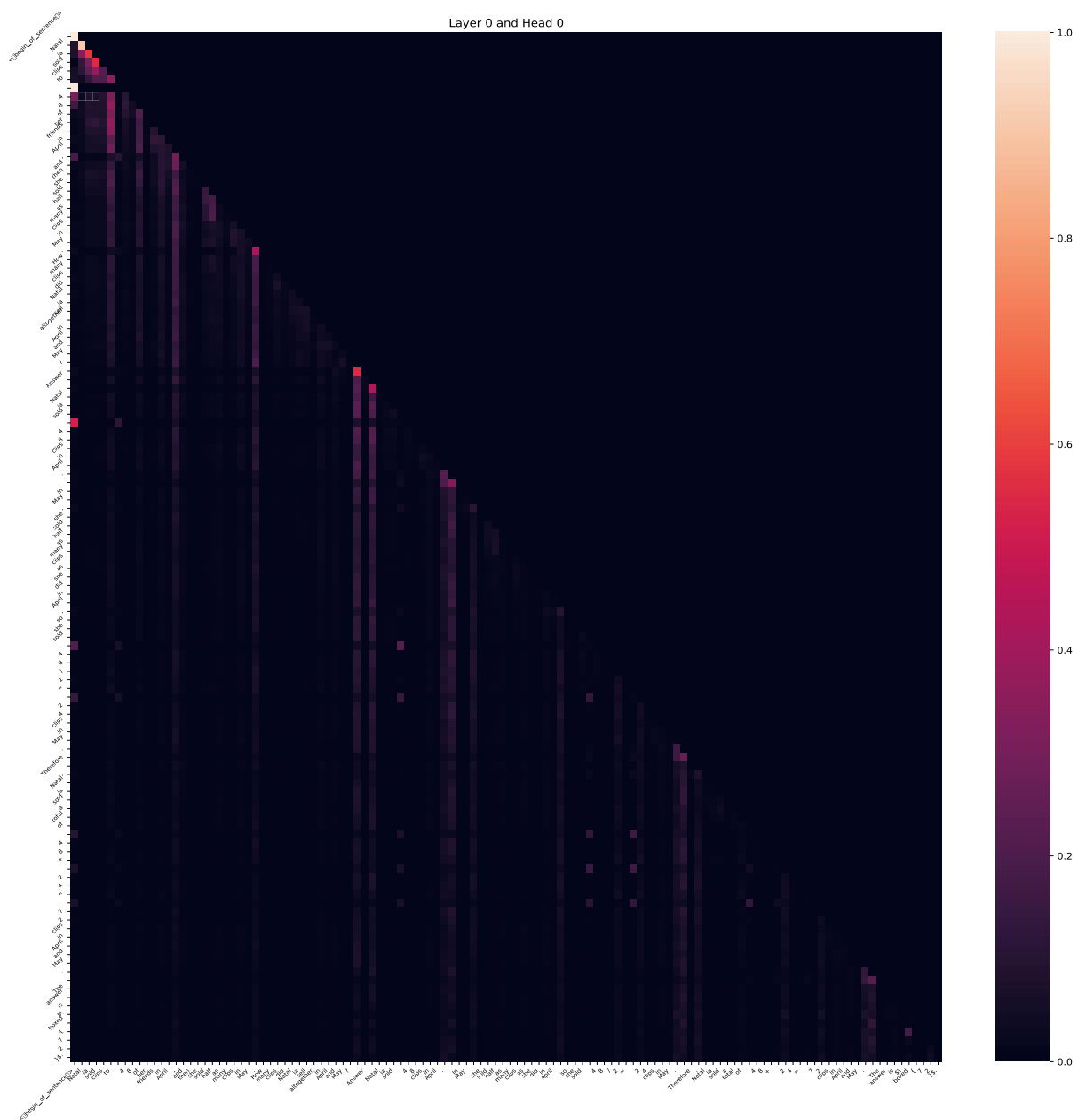


Figure 14. Llama-3-8B-Instruct 中注意力图的一个示例（第 0 层，第 0 个头）。

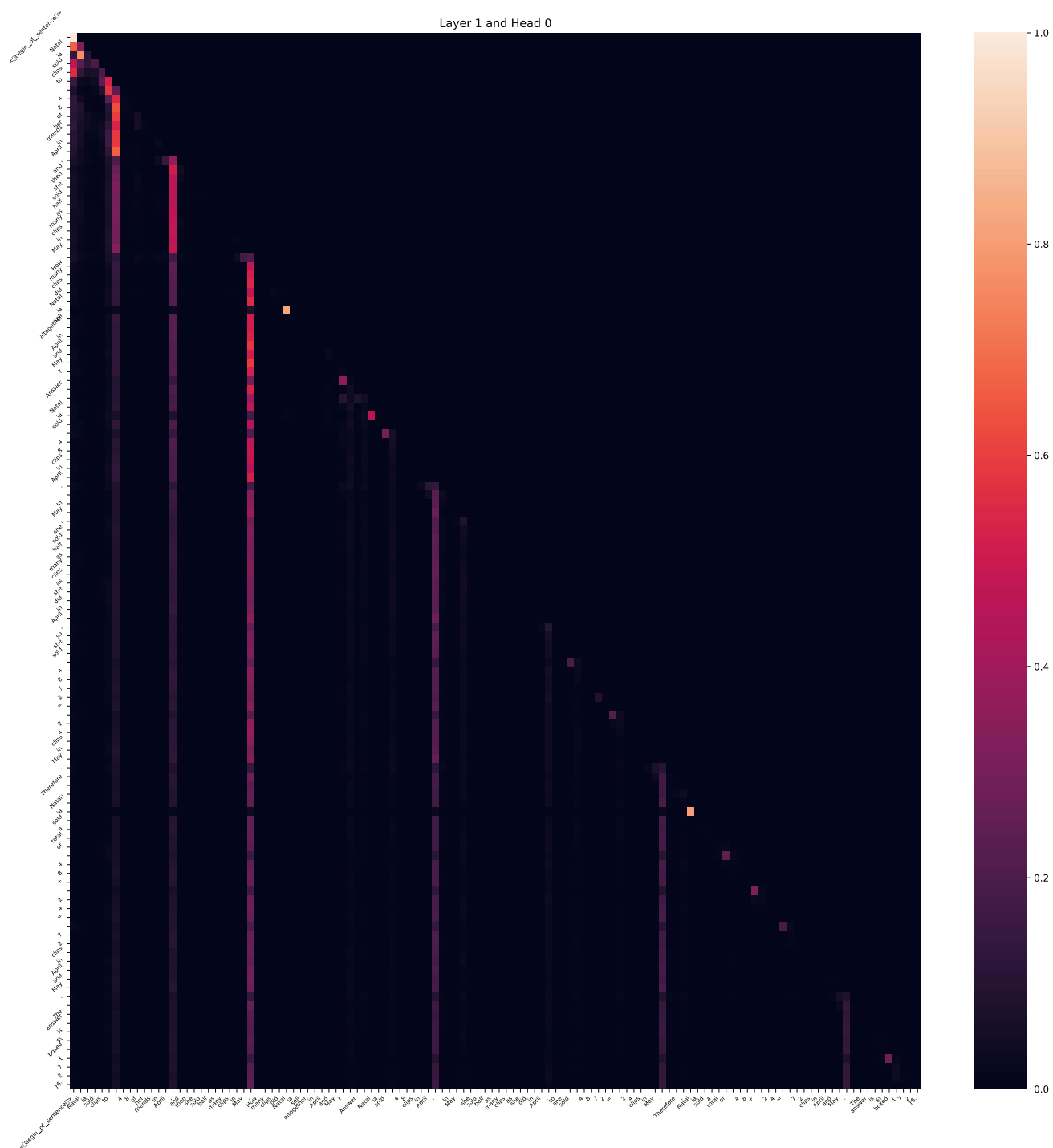


Figure 15. Llama-3-8B-Instruct 中注意力图的一个示例（第 1 层，第 0 个头）。

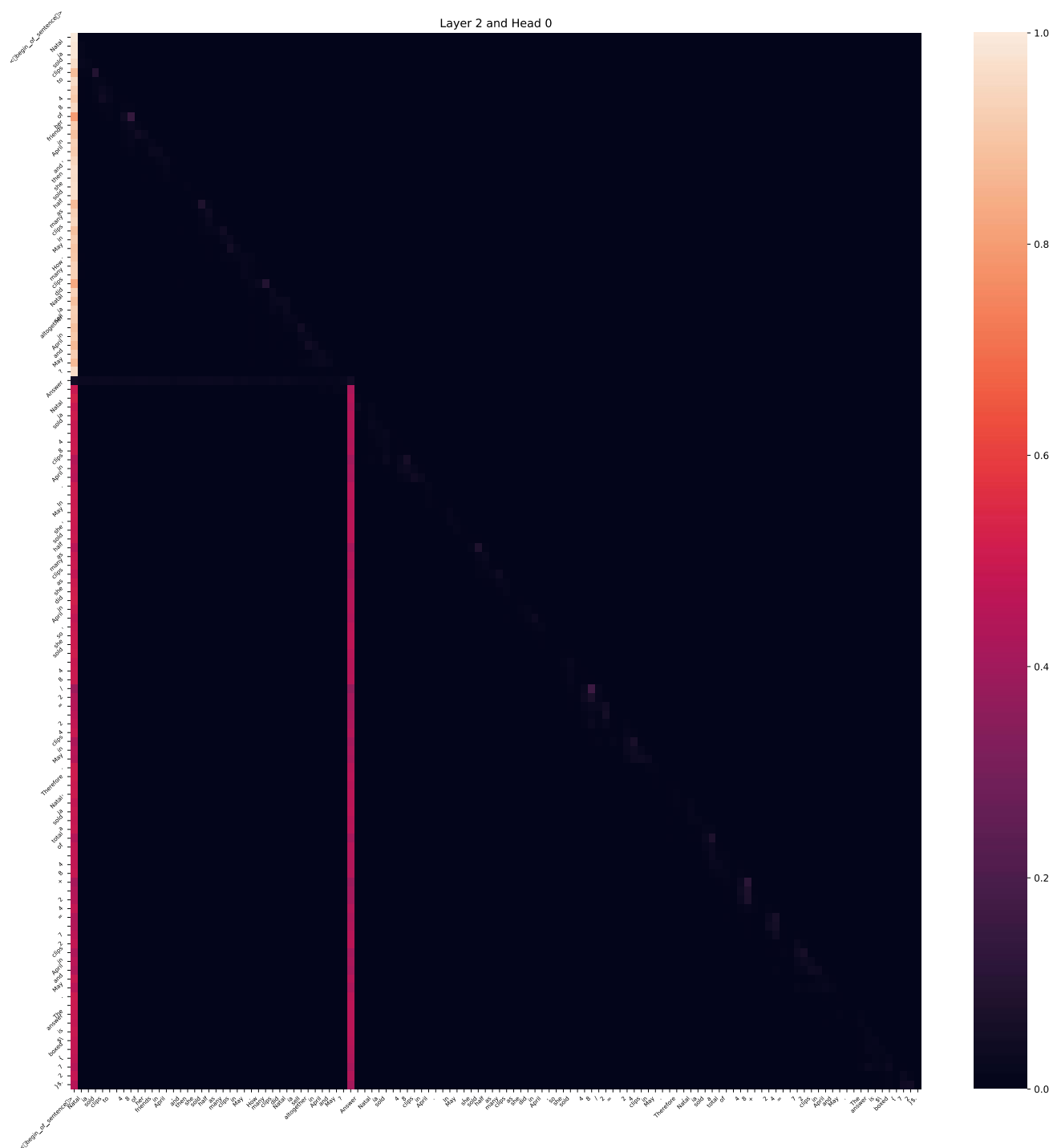


Figure 16. Llama-3-8B-Instruct 中注意力图的一个示例（第 2 层，第 0 个头）。